

Image Alignment and Stitching

Computer Vision: CS 566

Computer Science

University of Wisconsin-Madison

Image Alignment and Stitching

Combine multiple photos to create a larger photo

Topics:

- (1) Image Transformations
- (2) Computing Transformations
- (3) Warping Images
- (4) Blending Images



With the Cassini satellite's wide-angle camera aimed at Saturn, Cassini was able to capture 323 images in just over four hours in 2013. This final panorama used 141 of those images taken using red, green and blue spectral filters.

<http://www.jpl.nasa.gov/spaceimages/details.php?id=pia17172>

www.gigapan.com

Image Stitching Process

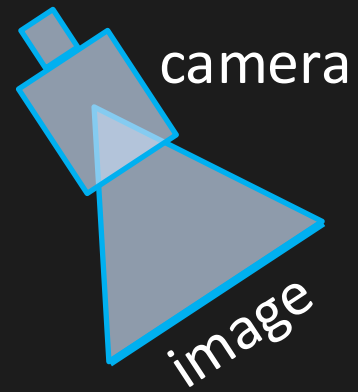
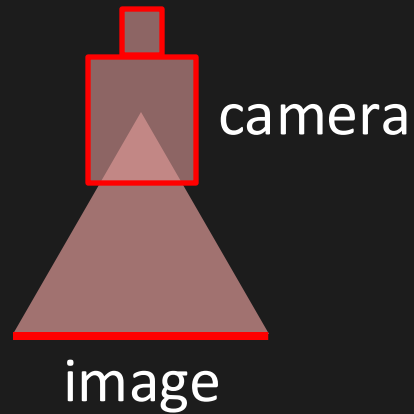
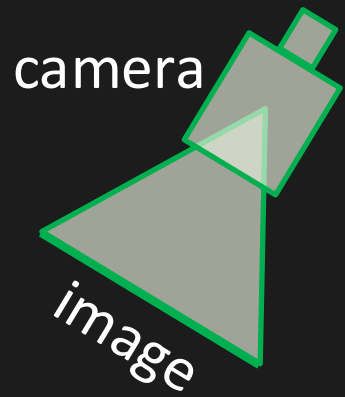
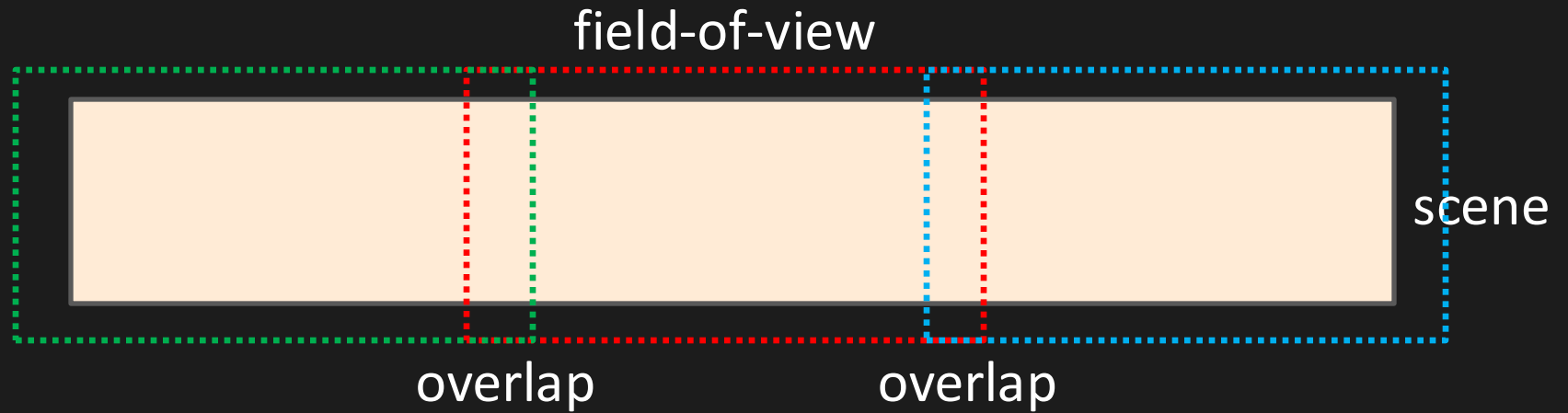
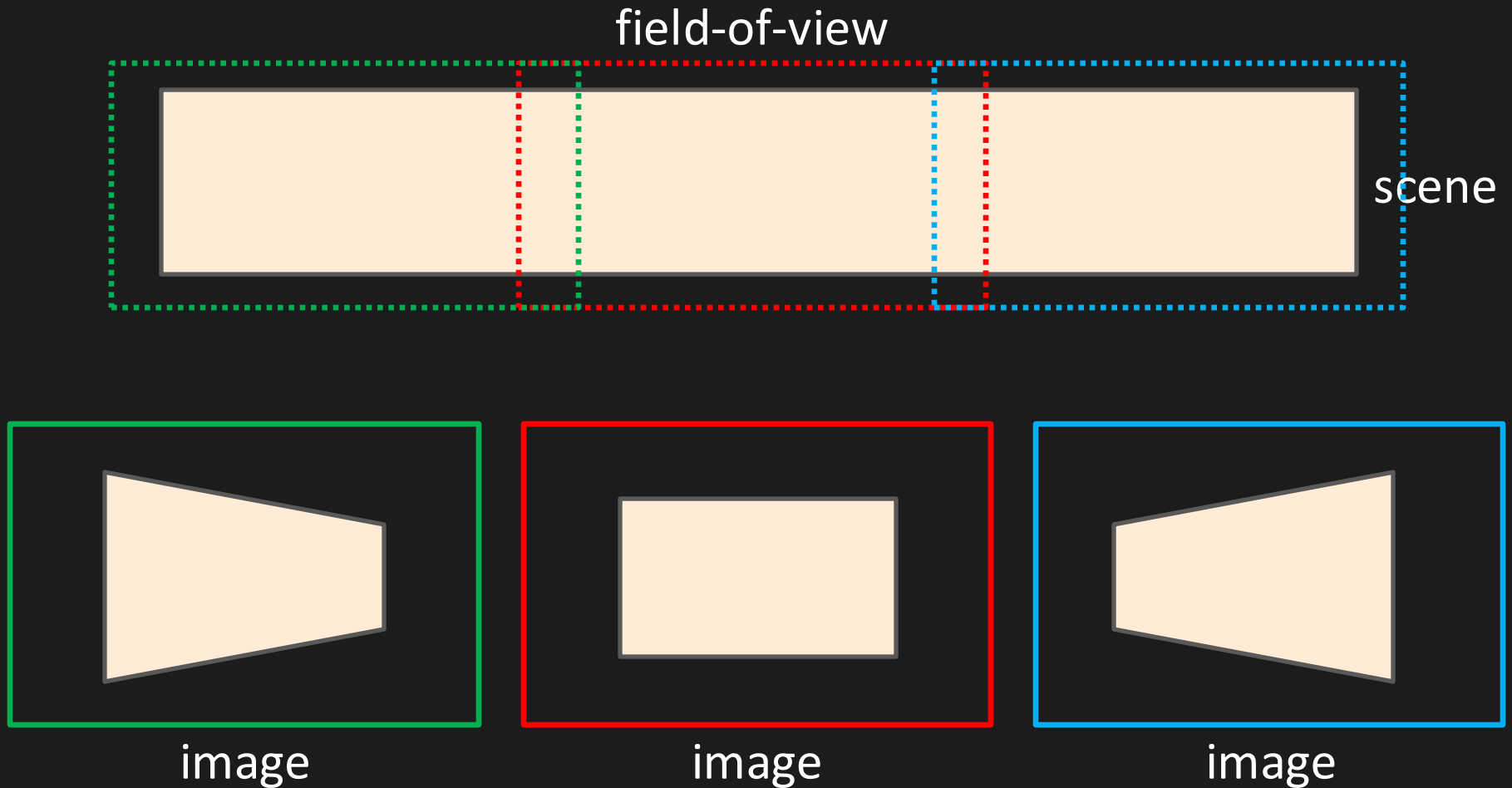
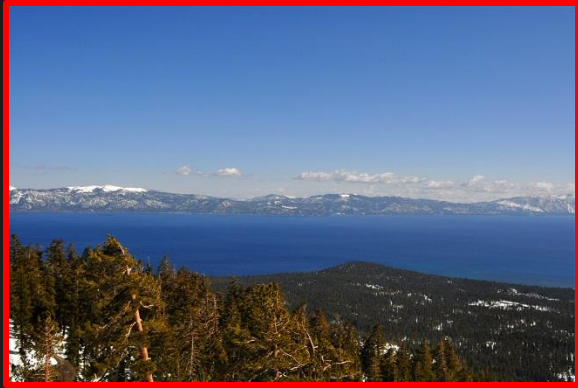


Image Stitching Process

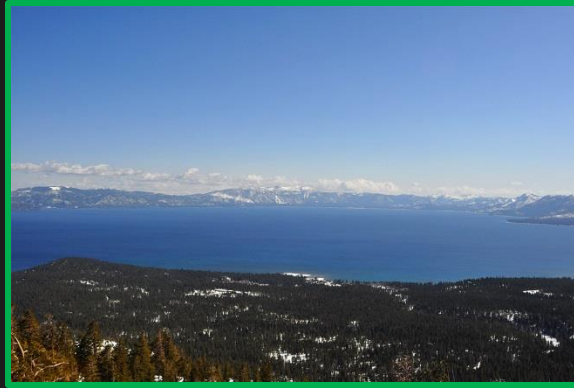


Overlap, but not Aligned

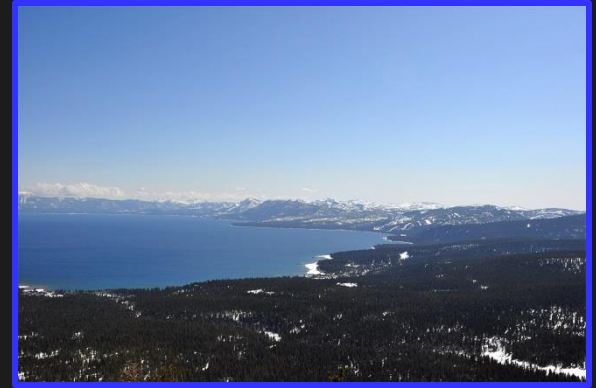
Image Stitching Process



Source Image 1



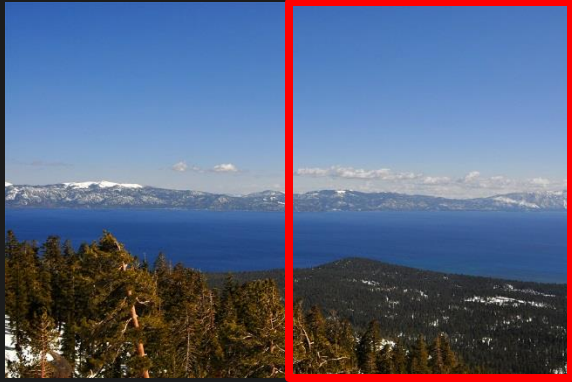
Source Image 2



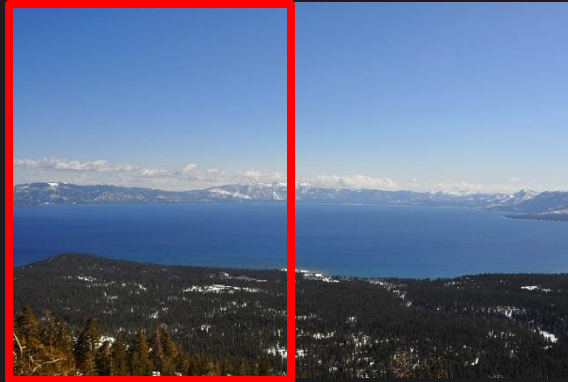
Source Image 3

How would you align these images?

Image Stitching Process



Source Image 1



Source Image 2



Source Image 3

How would you align these images?

Image Warping

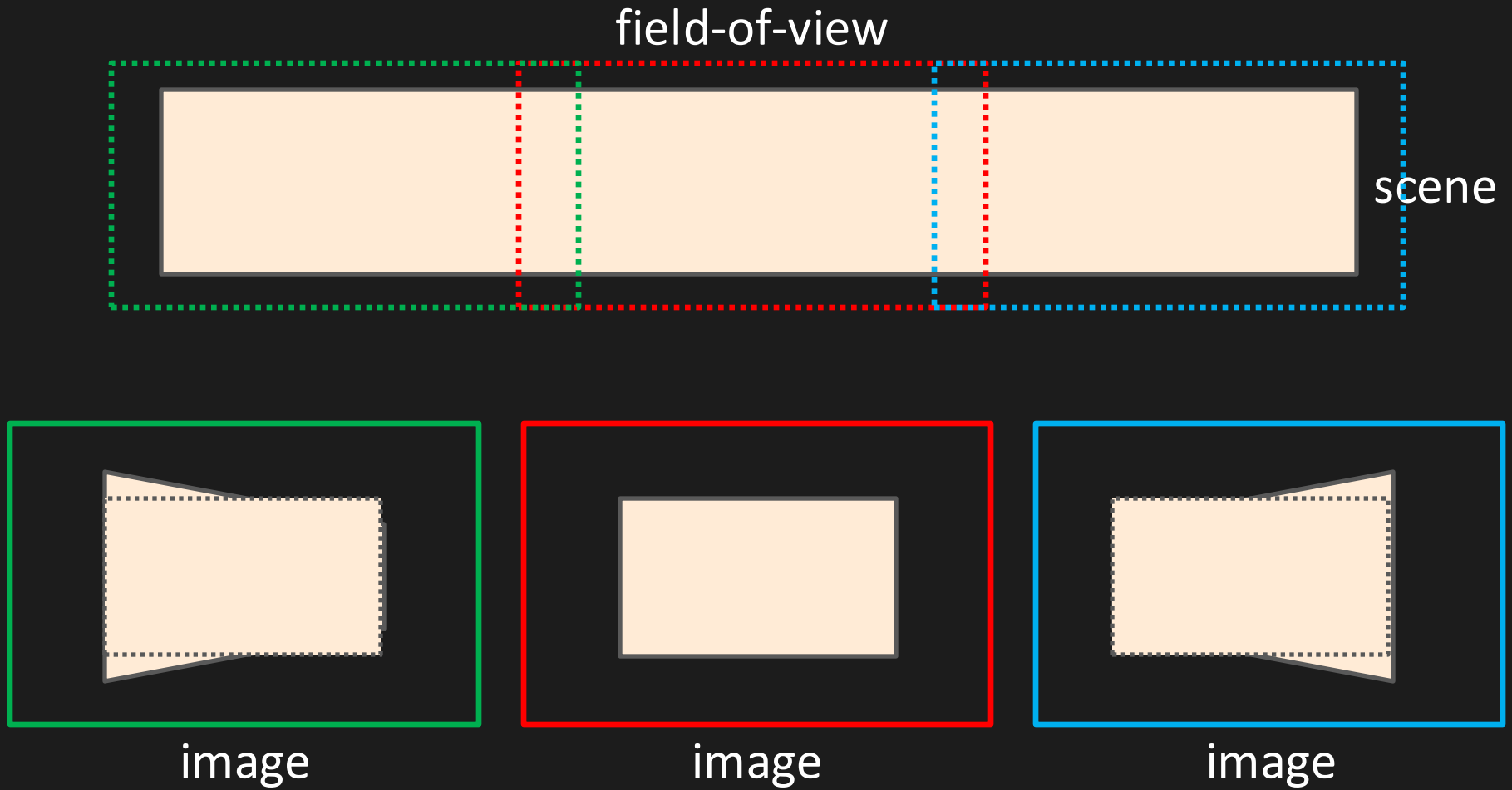


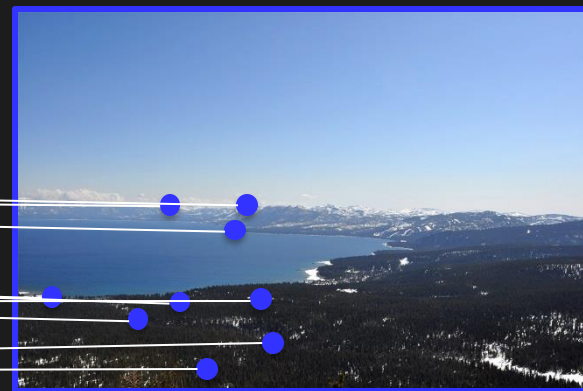
Image Stitching Process



Source Image 1



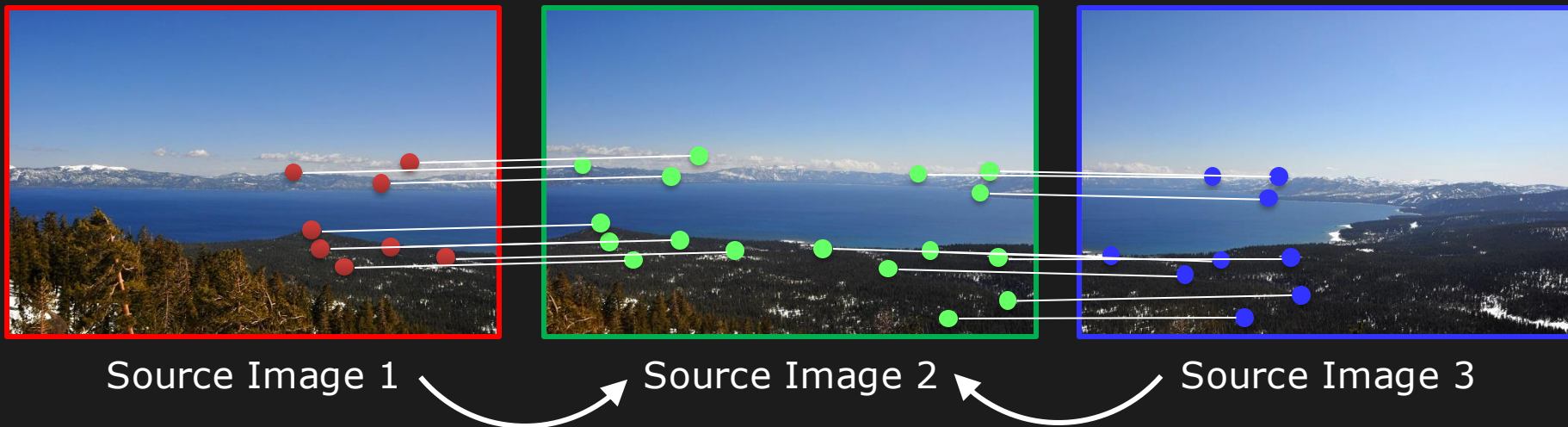
Source Image 2



Source Image 3

Find corresponding points
(using feature detectors like SIFT)

Image Stitching Process



Find geometric relationship between the corresponding points and therefore the images.

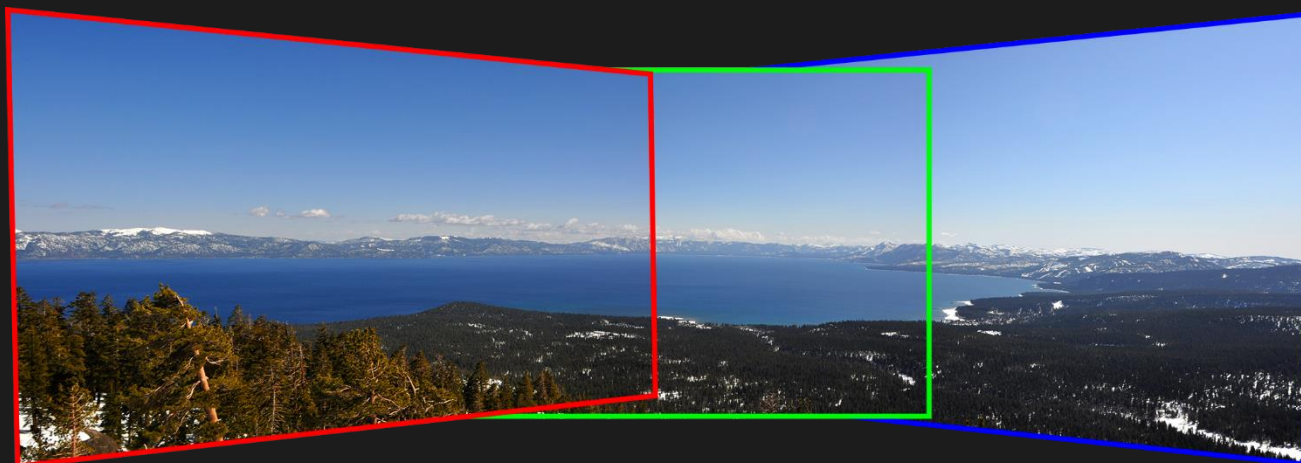
Image Stitching Process



Source Image 1

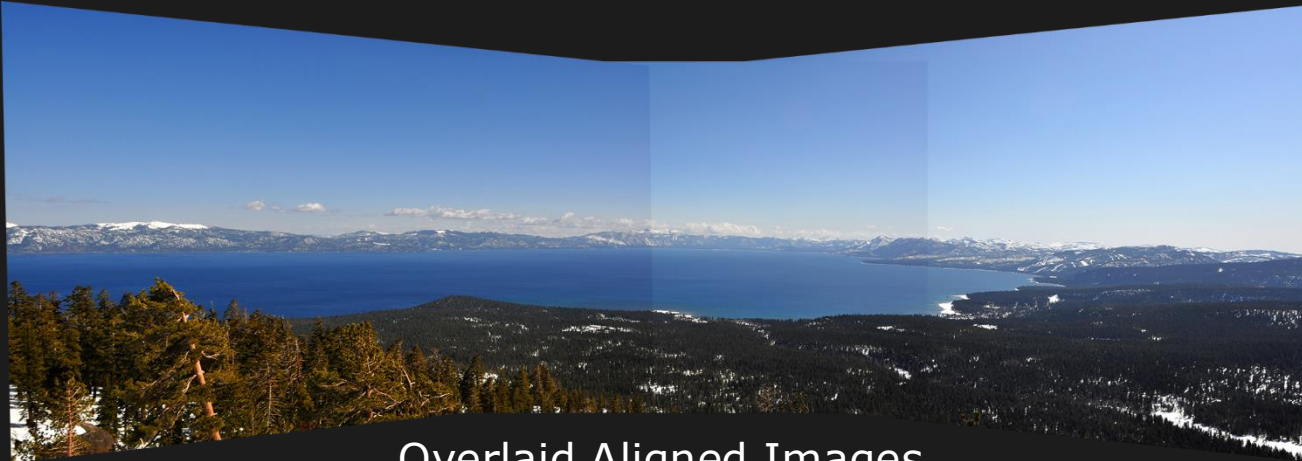
Source Image 2

Source Image 3

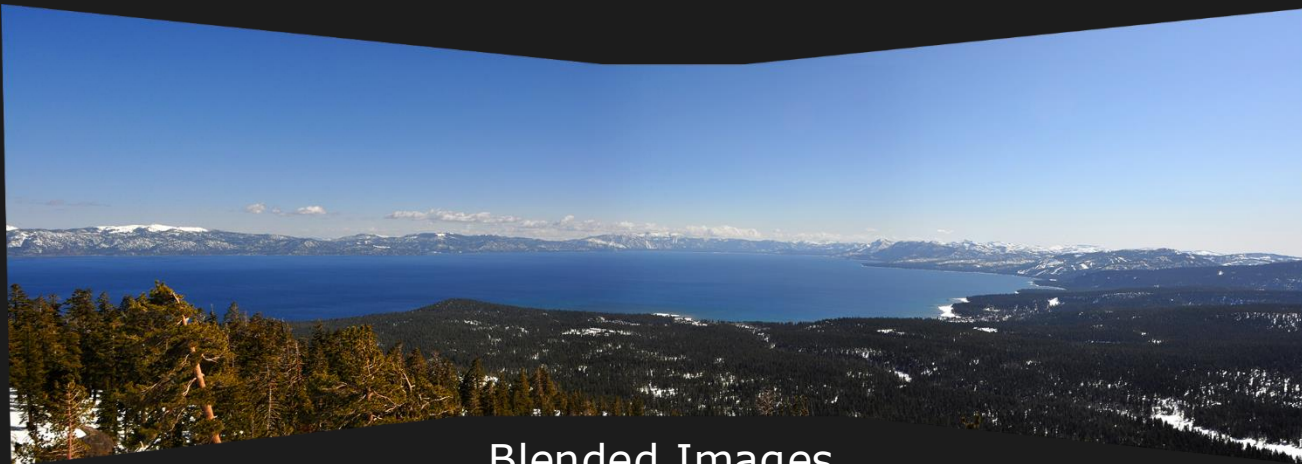


Warp images so that corresponding points align.

Image Stitching Process



Overlaid Aligned Images



Blended Images

Blend images to remove hard seams

Image Manipulation

Image Filtering: Change range (brightness) of image

$$g(x, y) = T_r(f(x, y))$$

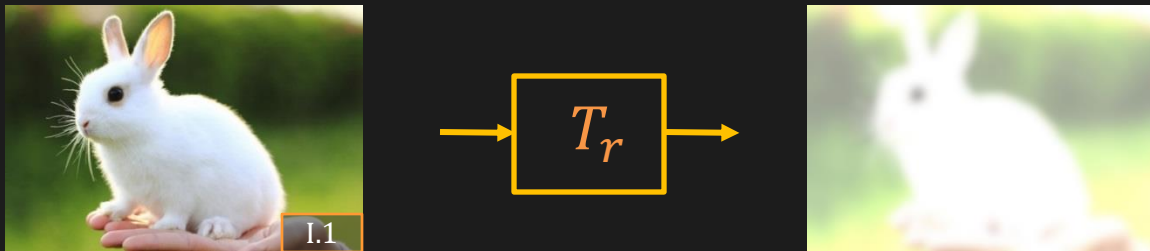
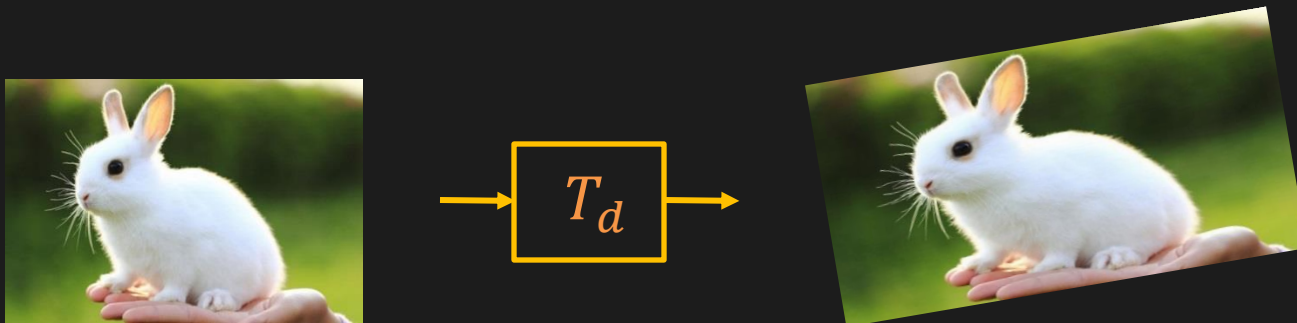


Image Warping: Change domain (location) of image

$$g(x, y) = f(T_d(x, y))$$

Transformation T_d is a coordinate changing operator.



Global Warping/Transformation



Translation



Rotation



Scaling and Aspect

$$g(x, y) = f(T(x, y))$$



Affine



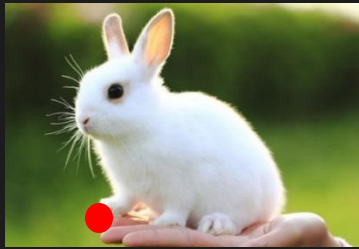
Perspective



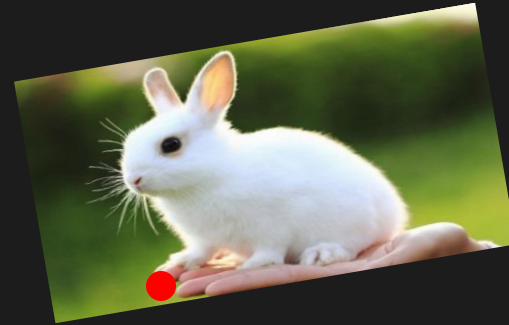
Barrel

Transformation T is the same over entire domain.
Often can be described by just a few parameters.

Linear Transformations



$$\mathbf{p}_1 = (x_1, y_1)$$



$$\mathbf{p}_2 = (x_2, y_2)$$

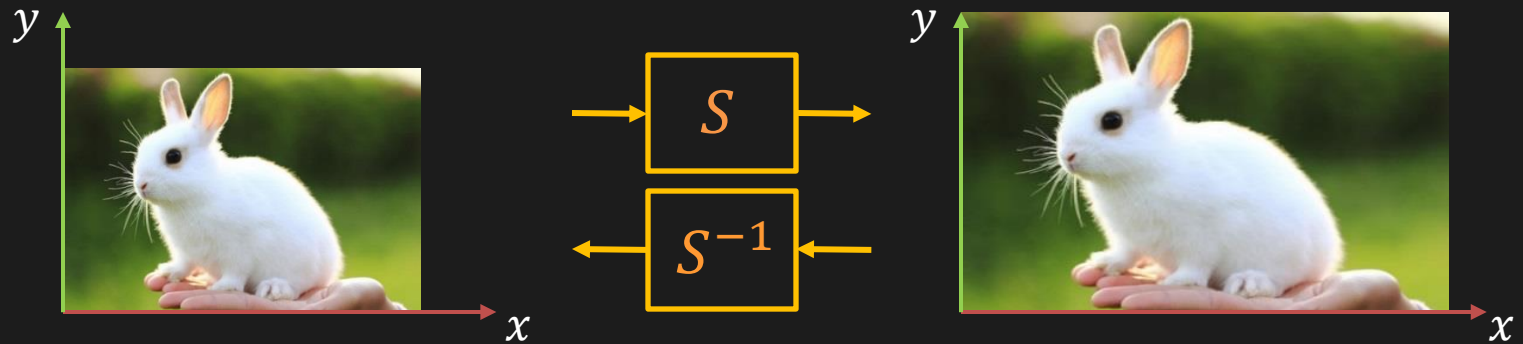
T can be represented by a matrix.

$$\mathbf{p}_2 = T\mathbf{p}_1$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = T \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Scaling (Stretching or Squishing)



Forward:

$$x_2 = ax_1 \quad y_2 = by_1$$

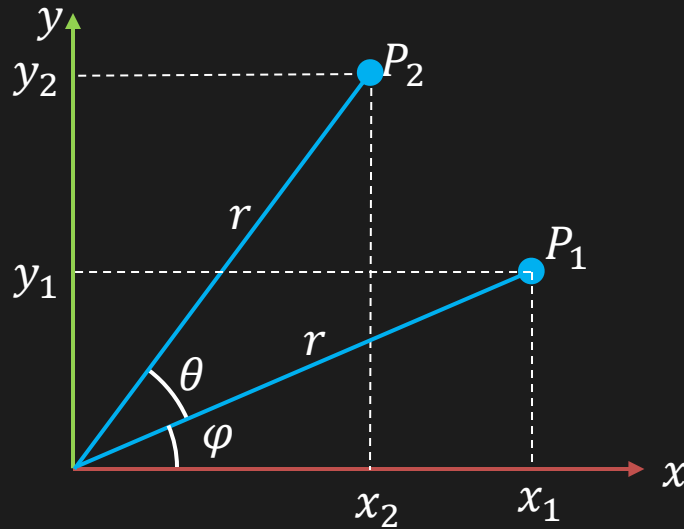
$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = S \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Inverse:

$$x_1 = \frac{1}{a}x_2 \quad y_1 = \frac{1}{b}y_2$$

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = S^{-1} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1/a & 0 \\ 0 & 1/b \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

2D Rotation



$$x_1 = r \cos(\varphi)$$

$$y_1 = r \sin(\varphi)$$

$$x_2 = r \cos(\varphi + \theta)$$

$$x_2 = r \cos \varphi \cos \theta - r \sin \varphi \sin \theta$$

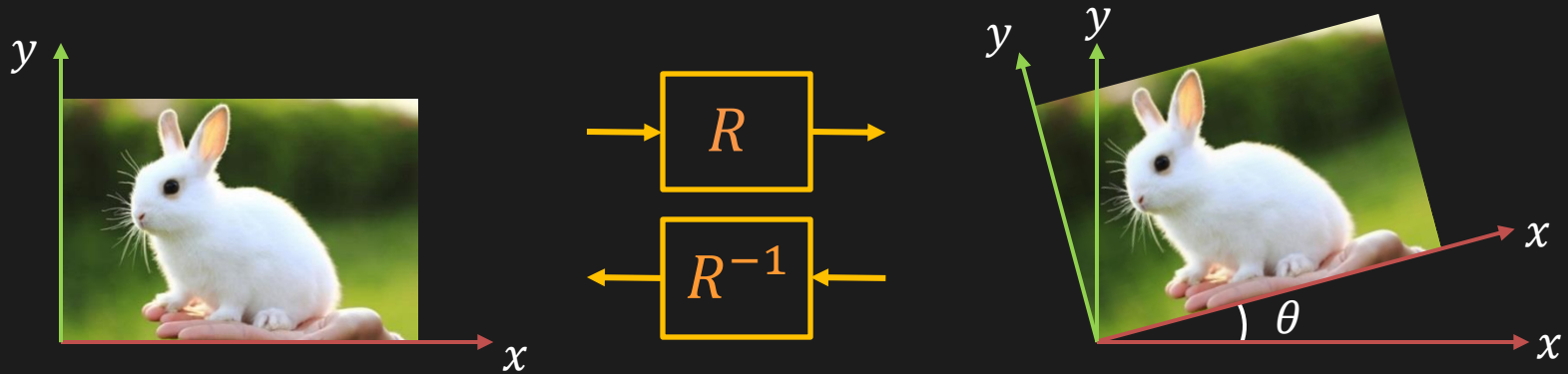
$$x_2 = x_1 \cos \theta - y_1 \sin \theta$$

$$y_2 = r \sin(\varphi + \theta)$$

$$y_2 = r \cos \varphi \sin \theta + r \sin \varphi \cos \theta$$

$$y_2 = x_1 \sin \theta + y_1 \cos \theta$$

Rotation



Forward:

$$x_2 = x_1 \cos \theta - y_1 \sin \theta$$

$$y_2 = x_1 \sin \theta + y_1 \cos \theta$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = R \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

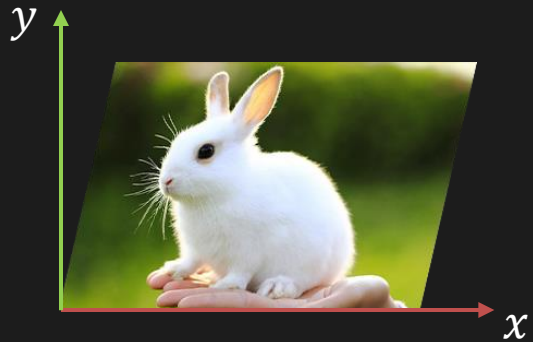
Inverse:

$$x_1 = x_2 \cos \theta + y_2 \sin \theta$$

$$y_1 = -x_2 \sin \theta + y_2 \cos \theta$$

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = R^{-1} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

Skew



Horizontal Skew:

$$x_2 = x_1 + m_x y_1$$

$$y_2 = y_1$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = S_x \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} 1 & m_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$



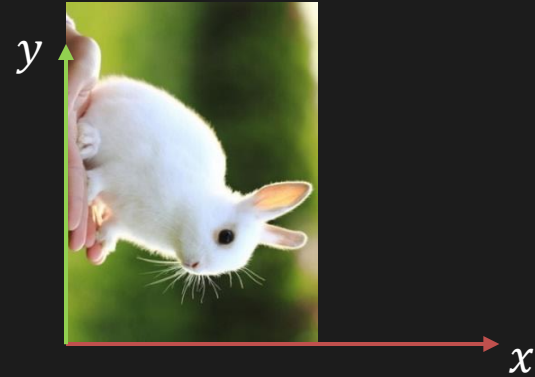
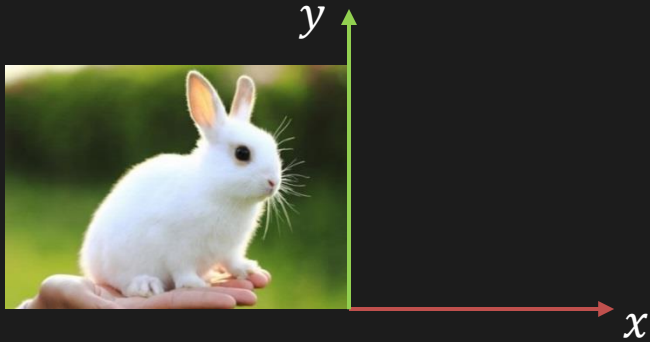
Vertical Skew:

$$x_2 = x_1$$

$$y_2 = m_y x_1 + y_1$$

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = S_y \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ m_y & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Mirror



Mirror about Y-axis:

$$x_2 = -x_1$$

$$y_2 = y_1$$

$$M_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Mirror about line $y = x$:

$$x_2 = y_1$$

$$y_2 = x_1$$

$$M_{xy} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

2x2 Matrix Transformations

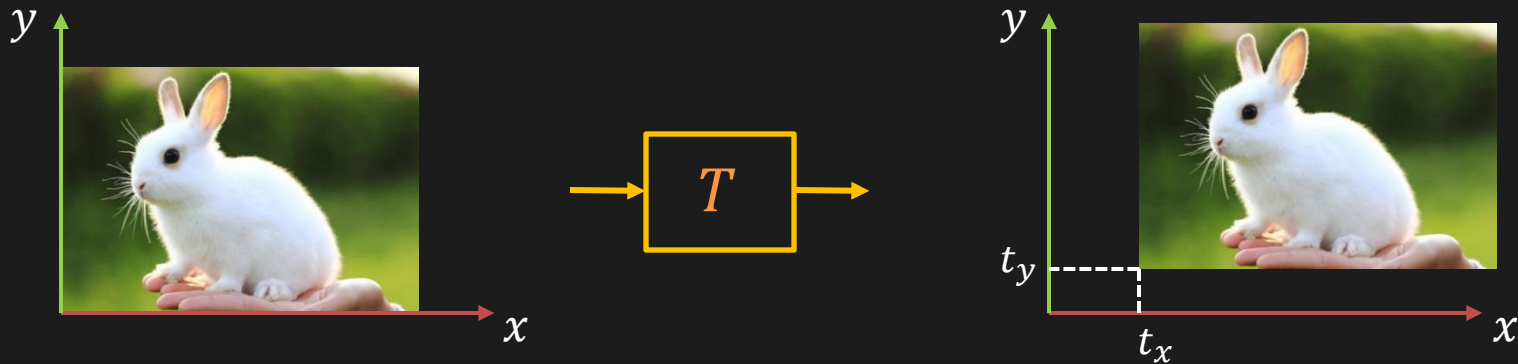
Any transformation of the form:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

- Origin maps to the origin
- Lines map to lines
- Parallel lines remain parallel
- Closed under composition

$$\left. \begin{array}{l} \mathbf{p}_2 = T_{21}\mathbf{p}_1 \\ \mathbf{p}_3 = T_{32}\mathbf{p}_2 \\ \mathbf{p}_3 = T_{31}\mathbf{p}_1 \end{array} \right\} \mathbf{p}_3 = T_{32}\mathbf{p}_2 = T_{32}T_{21}\mathbf{p}_1 \Rightarrow T_{31} = T_{32}T_{21}$$

Translation



$$x_2 = x_1 + t_x \quad y_2 = y_1 + t_y$$

Can translation be expressed as a 2x2 matrix? **No.**

Homogenous Coordinates

The **homogenous** representation of a 2D point $\mathbf{p} = (x, y)$ is a 3D point $\tilde{\mathbf{p}} = (\tilde{x}, \tilde{y}, \tilde{z})$. The third coordinate $\tilde{z} \neq 0$ is fictitious such that:

$$x = \frac{\tilde{x}}{\tilde{z}} \quad y = \frac{\tilde{y}}{\tilde{z}}$$

$$\mathbf{p} \equiv \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{z}x \\ \tilde{z}y \\ \tilde{z} \end{bmatrix} \equiv \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \end{bmatrix} = \tilde{\mathbf{p}}$$

Homogeneous Coordinates

Converting **to** homogeneous coordinates:

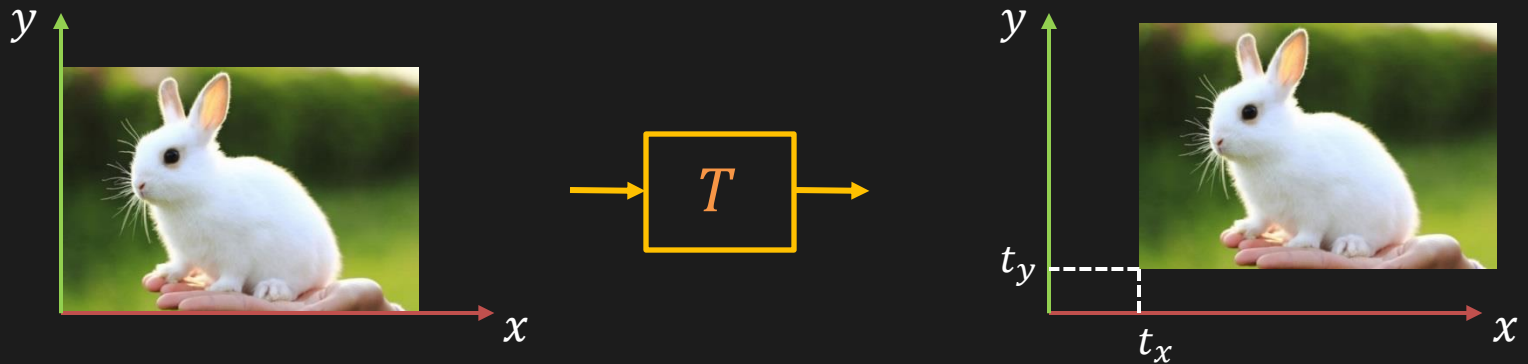
$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

*homogeneous image
coordinates*

Converting **from** homogeneous coordinates:

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

Translation



$$x_2 = x_1 + t_x$$

$$y_2 = y_1 + t_y$$

$$\begin{aligned}\tilde{x}_2 &= x_1 + t_x \\ \tilde{y}_2 &= y_1 + t_y \\ \tilde{z}_2 &= 1\end{aligned}$$

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Scaling, Rotation, Skew, Translation

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Scaling

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} 1 & m_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Skew

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Translation

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Rotation

Affine Transformation

Any transformation of the form:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$



Affine Transformation

Any transformation of the form:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$

- Origin does not necessarily map to the origin
- Lines map to lines
- Parallel lines remain parallel
- Closed under composition

Projective Transformation

Any transformation of the form:

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix} \quad \tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1$$

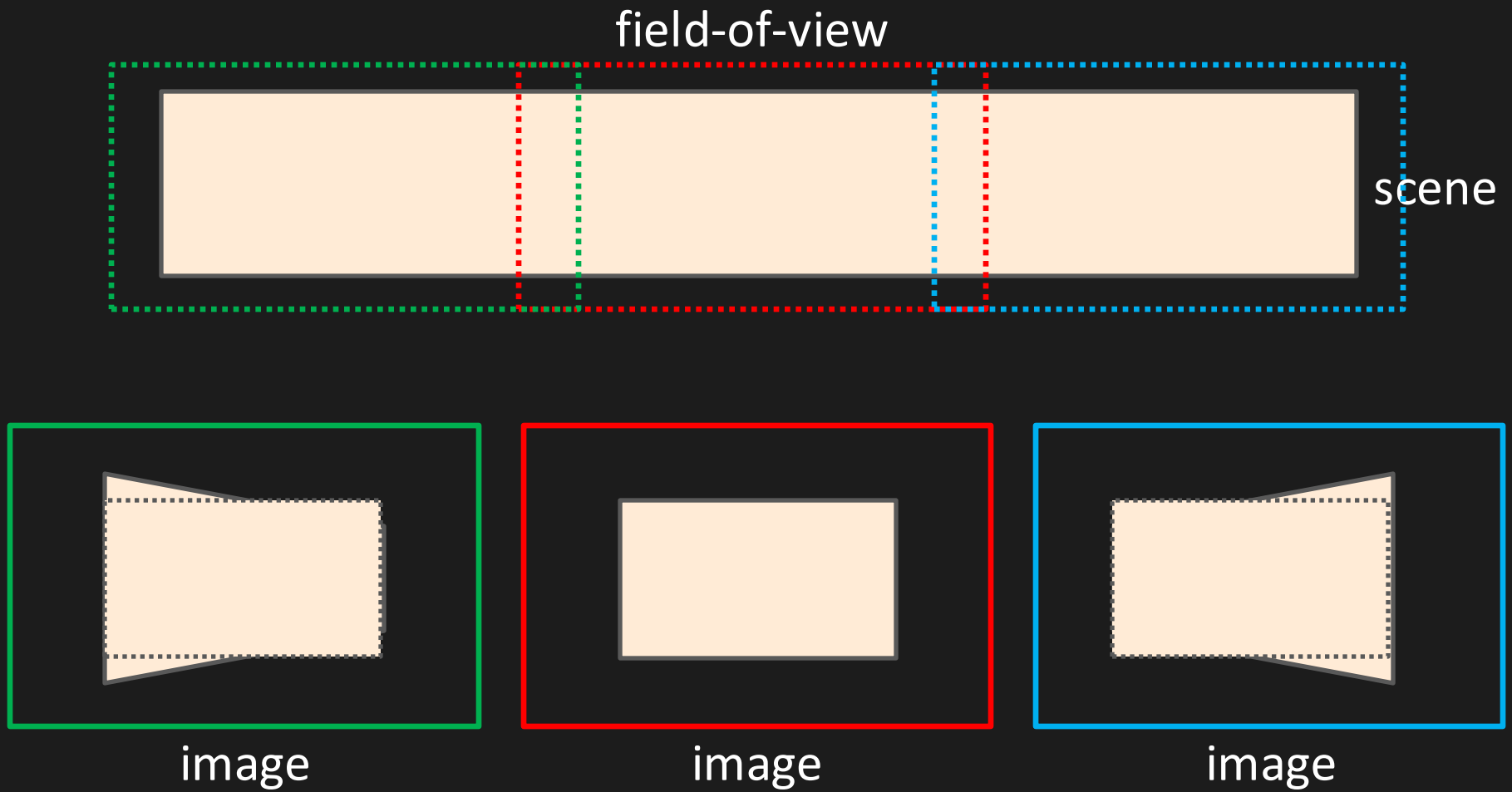


Also called **Homography**.

Projective Transformation

- Origin does not necessarily map to the origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Closed under composition

Image Warping

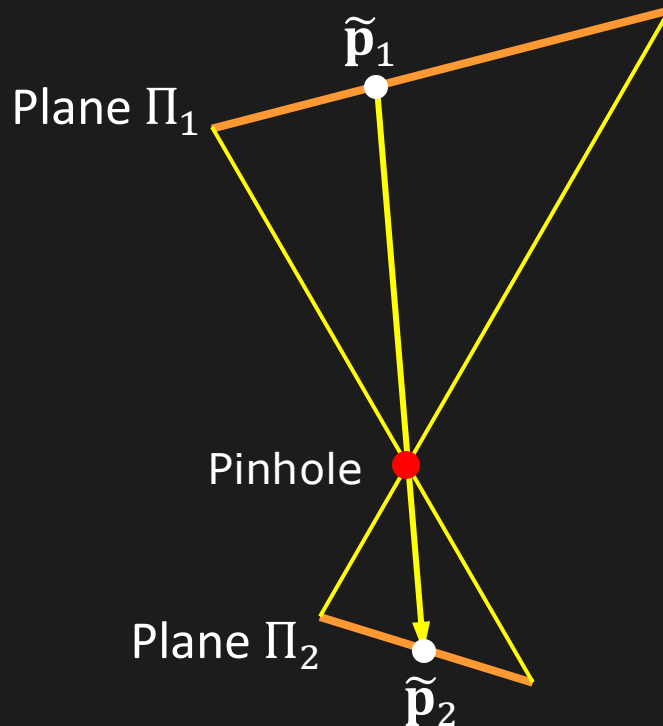


Remember Vanishing Points?



Projective Transformation

Mapping of one plane to another through a pinhole



$$\tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1$$

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$

Same as imaging a plane through a pinhole.

Example

Given homography

$$\mathbf{H} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 1 & 0 \\ 1 & 1 & 3 \end{bmatrix}$$

that maps points in image 2 into points in image 1,

where does the pixel at coordinates (10, 5) in image 2 project to in image 1?

Example

1. Convert point in image 2 to homogeneous coordinates: $p = (10, 5, 1)^T$

2. Compute $q = Hp$:

$$\begin{bmatrix} 1*10 + 2*5 + 3*1 = 23 \\ 4*10 + 1*5 + 0*1 = 45 \\ 1*10 + 1*5 + 3*1 = 18 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 1 & 0 \\ 1 & 1 & 3 \end{bmatrix} \begin{bmatrix} 10 \\ 5 \\ 1 \end{bmatrix}$$

3. Convert q to Cartesian coordinates:

$q = (sx, sy, s)^T = (23, 45, 18)^T$ or, in Cartesian coords,
 $(23/18, 45/18) = (1.28, 2.5)$ in image 1

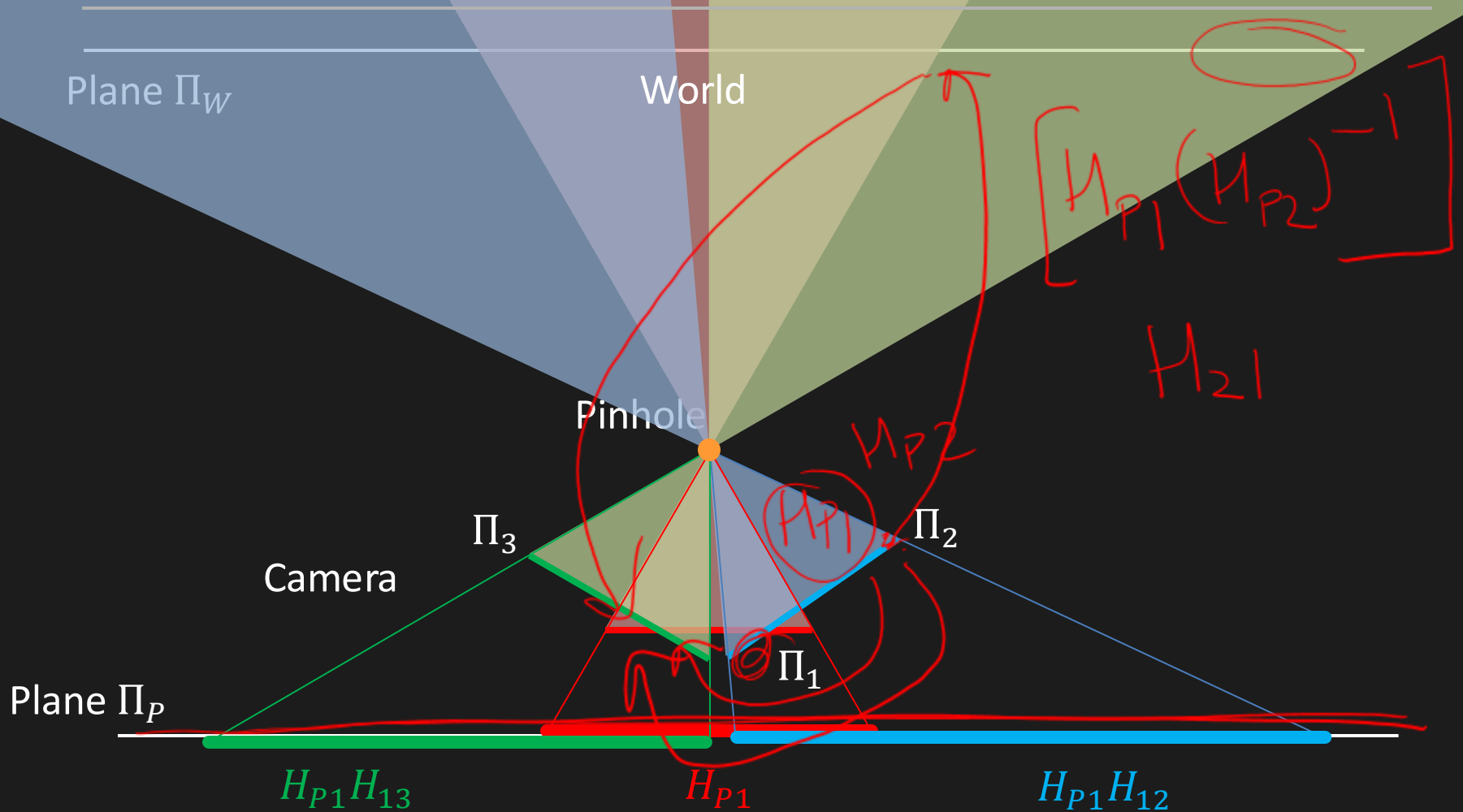
Projective Transformation

Homography can only be defined up to a scale.

$$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} \equiv k \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$

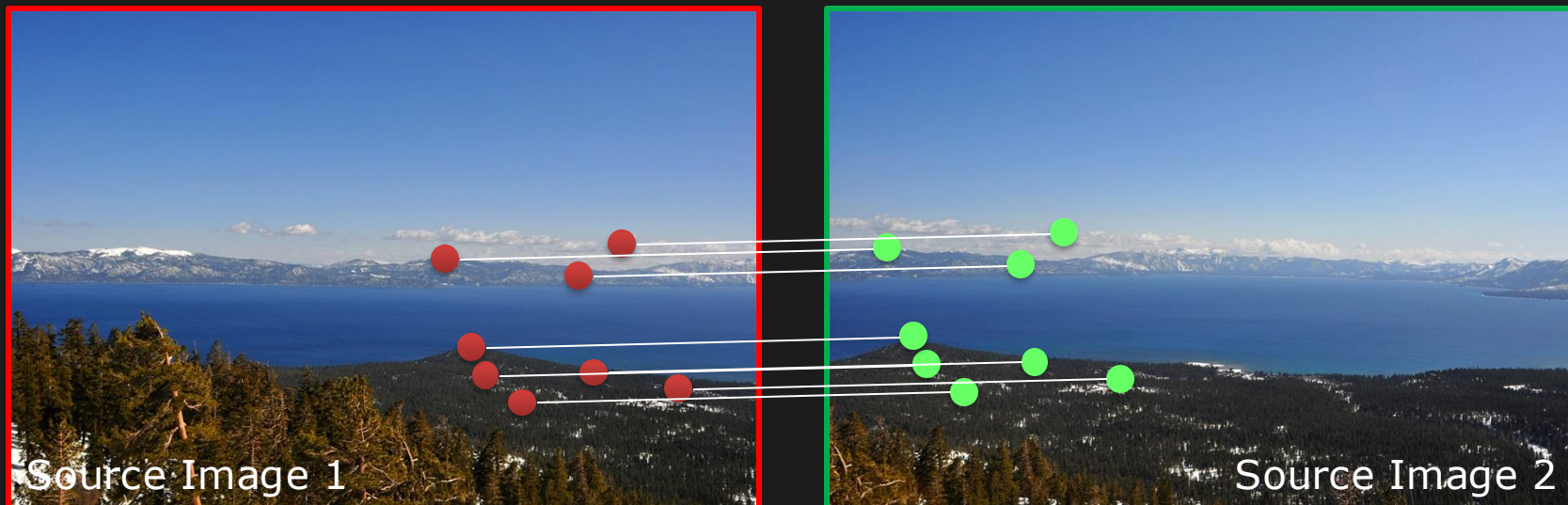
If we fix scale such that $\sqrt{\sum (h_{ij})^2} = 1$ then 8 free parameters

Homography Composition



Useful in stitching planar panoramas.

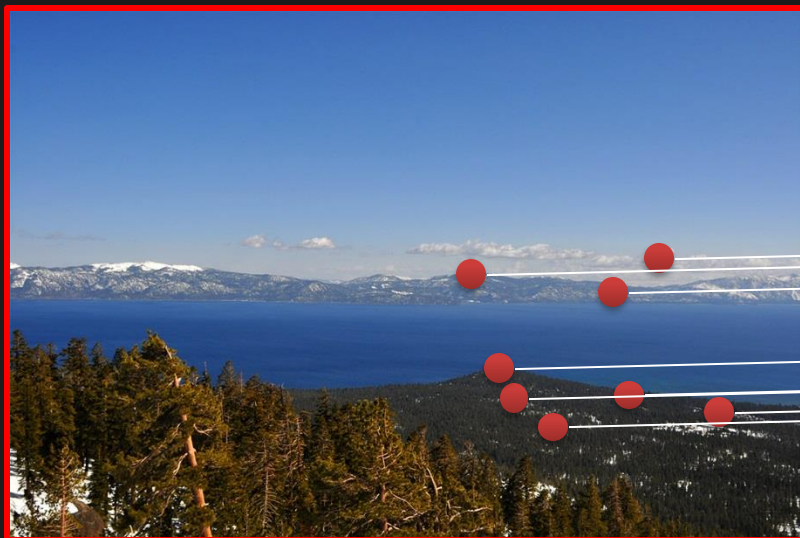
Computing Homography



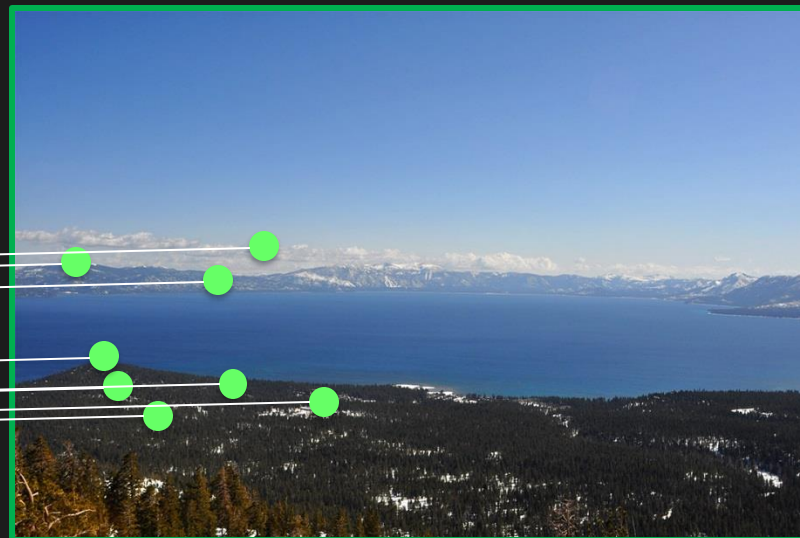
Given a set of matching features/points between images 1 and 2, find the homography H that best “agrees” with the matches.

The scene points should lie on a plane, or be distant (plane at infinity), or imaged from the same point.

Computing Homography



Source Image



Destination Image

$$\begin{bmatrix} x_d \\ y_d \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_d \\ \tilde{y}_d \\ \tilde{z}_d \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

How many unknowns? 9 ...But 8 degrees of freedom

How many minimum pairs of matching points? 4

Computing Homography

For a given pair i of corresponding points:

$$x_d^{(i)} = \frac{\tilde{x}_d^{(i)}}{\tilde{z}_d^{(i)}} = \frac{h_{11}x_s^{(i)} + h_{12}y_s^{(i)} + h_{13}}{h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}}$$



$$y_d^{(i)} = \frac{\tilde{y}_d^{(i)}}{\tilde{z}_d^{(i)}} = \frac{h_{21}x_s^{(i)} + h_{22}y_s^{(i)} + h_{23}}{h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}}$$

Rearranging the terms:

$$x_d^{(i)} \left(h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33} \right) = h_{11}x_s^{(i)} + h_{12}y_s^{(i)} + h_{13}$$



$$y_d^{(i)} \left(h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33} \right) = h_{21}x_s^{(i)} + h_{22}y_s^{(i)} + h_{23}$$

Computing Homography

For a given pair i of corresponding points:

$$x_d^{(i)} = \frac{\tilde{x}_d^{(i)}}{\tilde{z}_d^{(i)}} = \frac{h_{11}x_s^{(i)} + h_{12}y_s^{(i)} + h_{13}}{h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}}$$



$$y_d^{(i)} = \frac{\tilde{y}_d^{(i)}}{\tilde{z}_d^{(i)}} = \frac{h_{21}x_s^{(i)} + h_{22}y_s^{(i)} + h_{23}}{h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}}$$

Rearranging the terms:

$$x_d^{(i)} (h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}) - h_{11}x_s^{(i)} - h_{12}y_s^{(i)} - h_{13} = 0$$



$$y_d^{(i)} (h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}) - h_{21}x_s^{(i)} - h_{22}y_s^{(i)} - h_{23} = 0$$

Computing Homography

$$x_d^{(i)} (h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}) - h_{11}x_s^{(i)} + h_{12}y_s^{(i)} + h_{13} = 0$$

$$y_d^{(i)} (h_{31}x_s^{(i)} + h_{32}y_s^{(i)} + h_{33}) - h_{21}x_s^{(i)} + h_{22}y_s^{(i)} + h_{23} = 0$$

Rearranging the terms and writing as linear equation:

$$\begin{bmatrix} x_s^{(i)} & y_s^{(i)} & 1 & 0 & 0 & 0 & -x_d^{(i)}x_s^{(i)} & -x_d^{(i)}y_s^{(i)} & -x_d^{(i)} \\ 0 & 0 & 0 & x_s^{(i)} & y_s^{(i)} & 1 & -y_d^{(i)}x_s^{(i)} & -y_d^{(i)}y_s^{(i)} & -y_d^{(i)} \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(Known)

h
(Unknown)

Computing Homography

Combining the equations for all corresponding points:

$$\begin{bmatrix}
 x_s^{(1)} & y_s^{(1)} & 1 & 0 & 0 & 0 & -x_d^{(1)}x_s^{(1)} & -x_d^{(1)}y_s^{(1)} & -x_d^{(1)} \\
 0 & 0 & 0 & x_s^{(1)} & y_s^{(1)} & 1 & -y_d^{(1)}x_s^{(1)} & -y_d^{(1)}y_s^{(1)} & -y_d^{(1)} \\
 & & & & & \vdots & & & \\
 x_s^{(i)} & y_s^{(i)} & 1 & 0 & 0 & 0 & -x_d^{(i)}x_s^{(i)} & -x_d^{(i)}y_s^{(i)} & -x_d^{(i)} \\
 0 & 0 & 0 & x_s^{(i)} & y_s^{(i)} & 1 & -y_d^{(i)}x_s^{(i)} & -y_d^{(i)}y_s^{(i)} & -y_d^{(i)} \\
 & & & & & \vdots & & & \\
 x_s^{(n)} & y_s^{(n)} & 1 & 0 & 0 & 0 & -x_d^{(n)}x_s^{(n)} & -x_d^{(n)}y_s^{(n)} & -x_d^{(n)} \\
 0 & 0 & 0 & x_s^{(n)} & y_s^{(n)} & 1 & -y_d^{(n)}x_s^{(n)} & -y_d^{(n)}y_s^{(n)} & -y_d^{(n)}
 \end{bmatrix}
 \begin{bmatrix}
 h_{11} \\
 h_{12} \\
 h_{13} \\
 h_{21} \\
 h_{22} \\
 h_{23} \\
 h_{31} \\
 h_{32} \\
 h_{33}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

A
(Known)

$\mathbf{h}$
(Unknown)

Solve for $\mathbf{h}$: $\boxed{A \mathbf{h} = \mathbf{0}}$ such that $\|\mathbf{h}\|^2 = 1$

Constrained Least Squares

Solve for $\mathbf{h}$: $A \mathbf{h} = \mathbf{0}$ such that $\|\mathbf{h}\|^2 = 1$

Define least squares problem:

$$\min_{\mathbf{h}} \|A\mathbf{h}\|^2 \text{ such that } \|\mathbf{h}\|^2 = 1$$

We know that:

$$\|A\mathbf{h}\|^2 = (A\mathbf{h})^T (A\mathbf{h}) = \mathbf{h}^T A^T A \mathbf{h} \quad \text{and} \quad \|\mathbf{h}\|^2 = \mathbf{h}^T \mathbf{h} = 1$$

$$\min_{\mathbf{h}} (\mathbf{h}^T A^T A \mathbf{h}) \text{ such that } \mathbf{h}^T \mathbf{h} = 1$$

Constrained Least Squares

$$\min_{\mathbf{h}} (\mathbf{h}^T A^T A \mathbf{h}) \text{ such that } \mathbf{h}^T \mathbf{h} = 1$$

Define Loss function $L(\mathbf{h}, \lambda)$:

$$L(\mathbf{h}, \lambda) = \mathbf{h}^T A^T A \mathbf{h} - \lambda(\mathbf{h}^T \mathbf{h} - 1)$$

Taking derivatives of $L(\mathbf{h}, \lambda)$ w.r.t $\mathbf{h}$: $2A^T A \mathbf{h} - 2\lambda \mathbf{h} = \mathbf{0}$

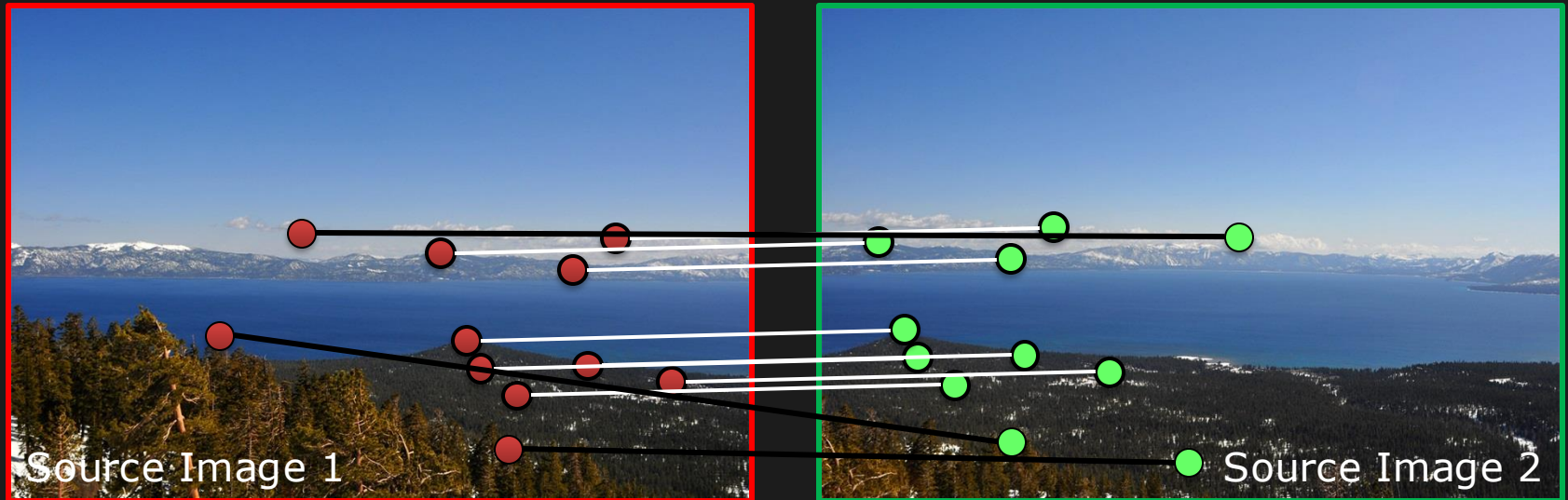
$$A^T A \mathbf{h} = \lambda \mathbf{h}$$

Eigenvalue Problem

Eigenvector $\mathbf{h}$ with smallest eigenvalue λ of matrix $A^T A$ minimizes the loss function $L(\mathbf{h})$.

Matlab: `eig(A' * A)` returns eigenvalues and vectors of $A^T A$

What Could Go Wrong?



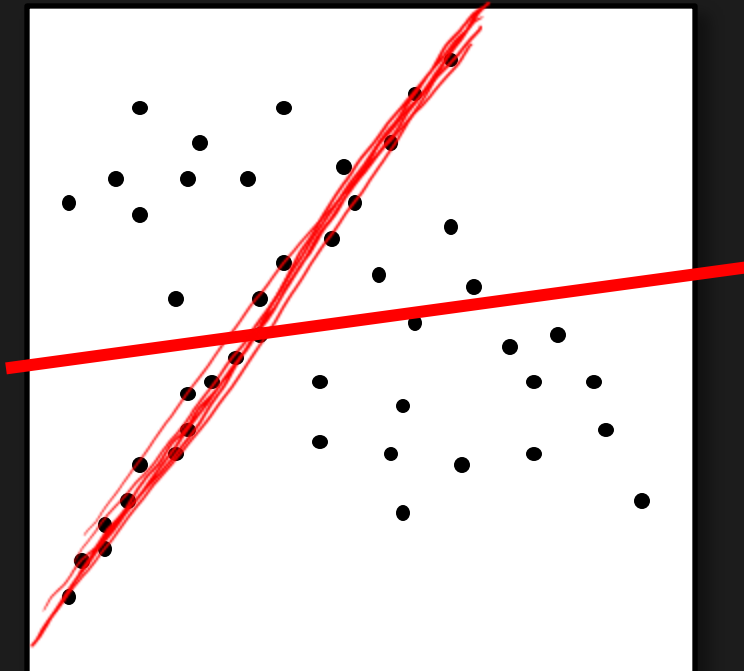
Outliers!

We need to robustly compute transformation in the presence of wrong matches.

If **number of outliers** $< 50\%$, then **RANSAC** to the rescue!

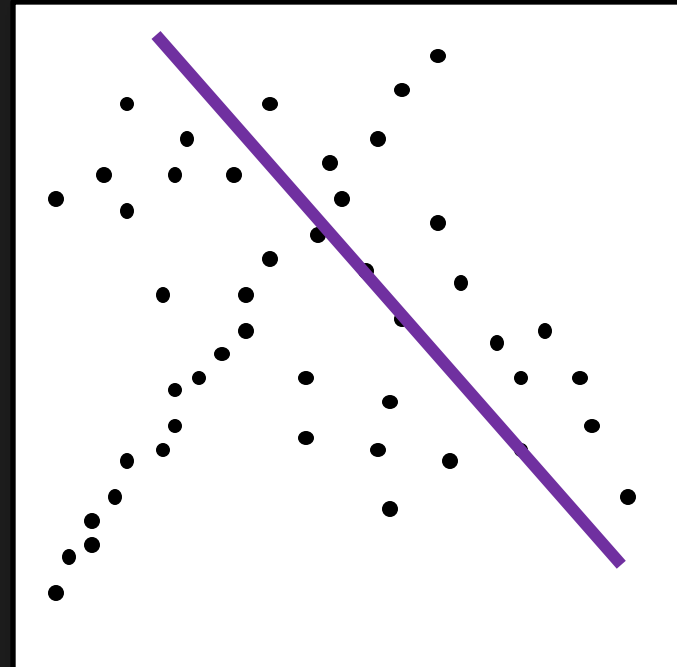
RANSAC Example: Line Fitting

Robust line fitting:



Least Squares Fitting

Inliers: 2

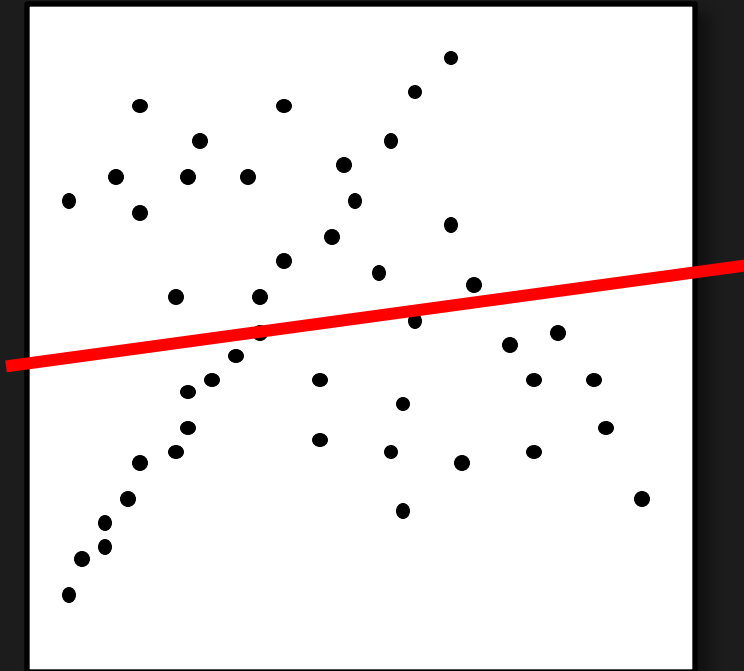


RANSAC Iteration 1

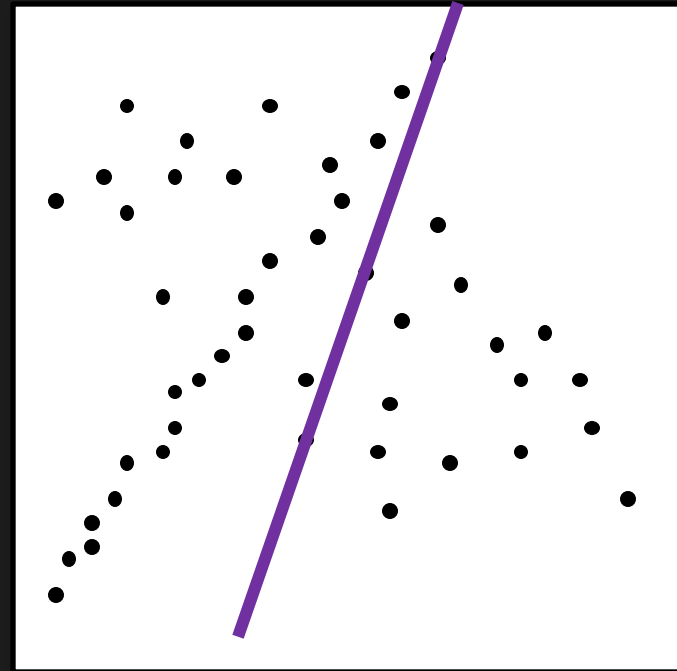
Inliers: 4

RANSAC Example: Line Fitting

Robust line fitting:



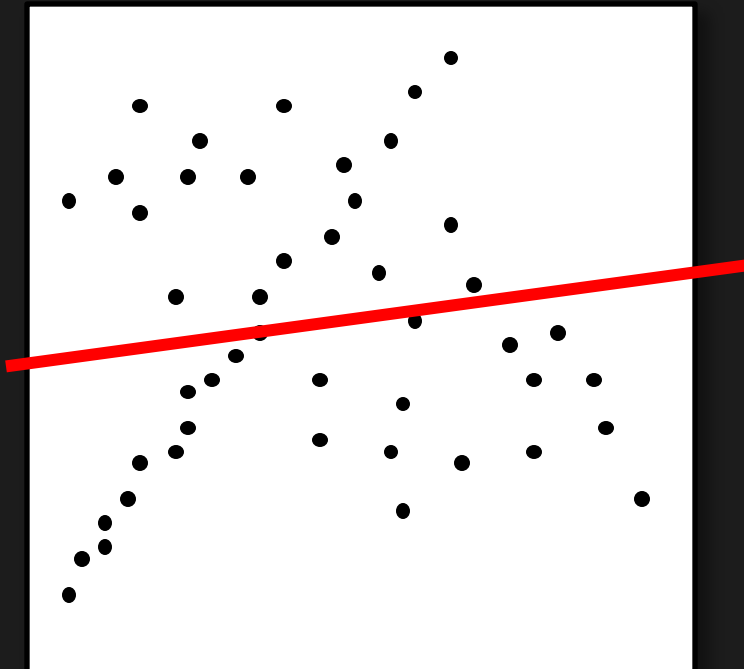
Least Squares Fitting
Inliers: 2



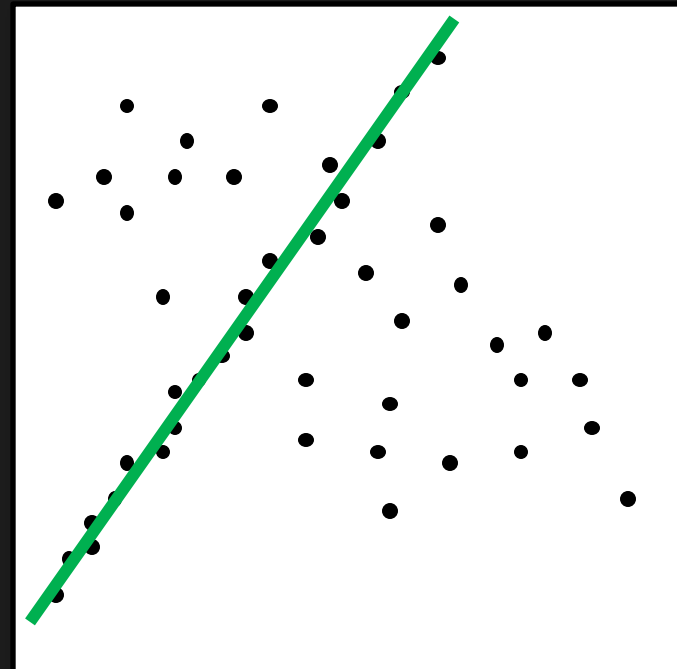
RANSAC Iteration 2
Inliers: 3

RANSAC Example: Line Fitting

Robust line fitting:



Least Squares Fitting
Inliers: 2



RANSAC Iteration i
Inliers: 20

RANdom SAmple Consensus

General RANSAC Algorithm:

1. Randomly choose s samples. Typically s is the minimum samples to fit a model.
2. Fit the model to the randomly chosen samples.
3. Count the number M of data points (inliers) that fit the model within a measure of error ϵ .
4. Repeat Steps 1-3 N times
5. Choose the model that has the largest number M of inliers.

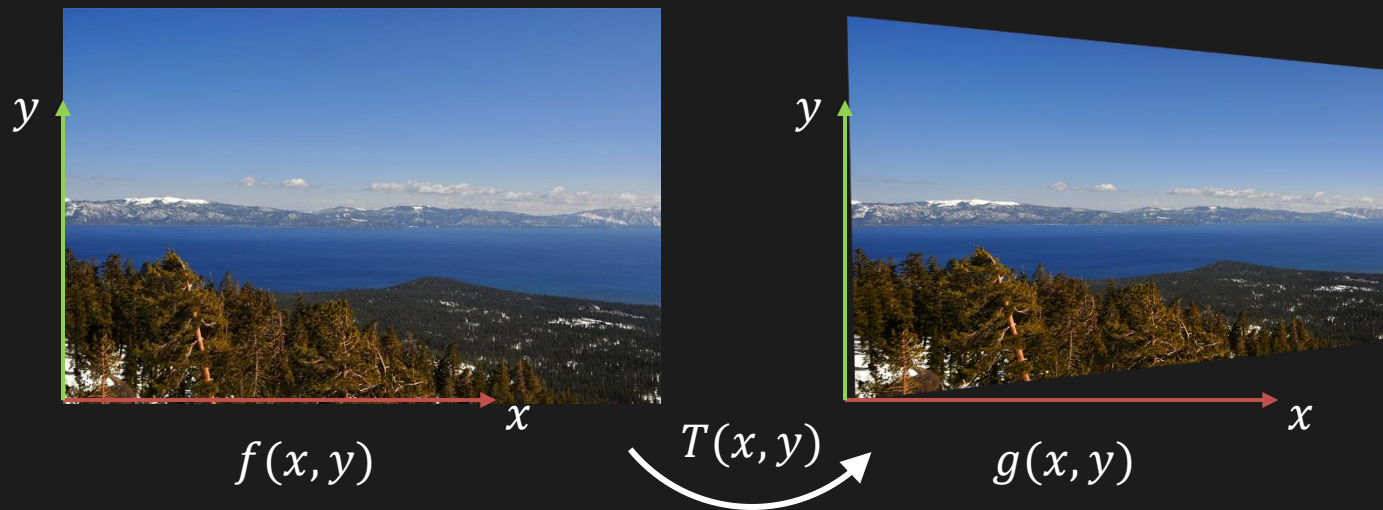
For homography:

$s = 4$ points. ϵ is acceptable alignment error in pixels.

Back to Warping Images

Given a transformation T and a source image $f(x, y)$, compute the transformed image $g(x, y)$

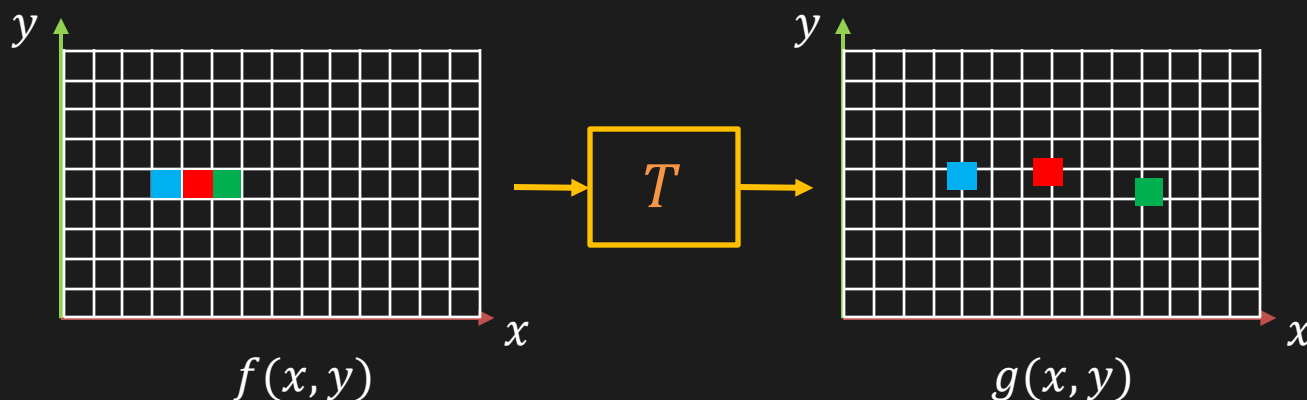
$$g(x, y) = f(T(x, y))$$



Forward Warping

Send each pixel (x, y) in $f(x, y)$ to its corresponding location $T(x, y)$ in $g(x, y)$

$$g(x, y) = f(T(x, y))$$



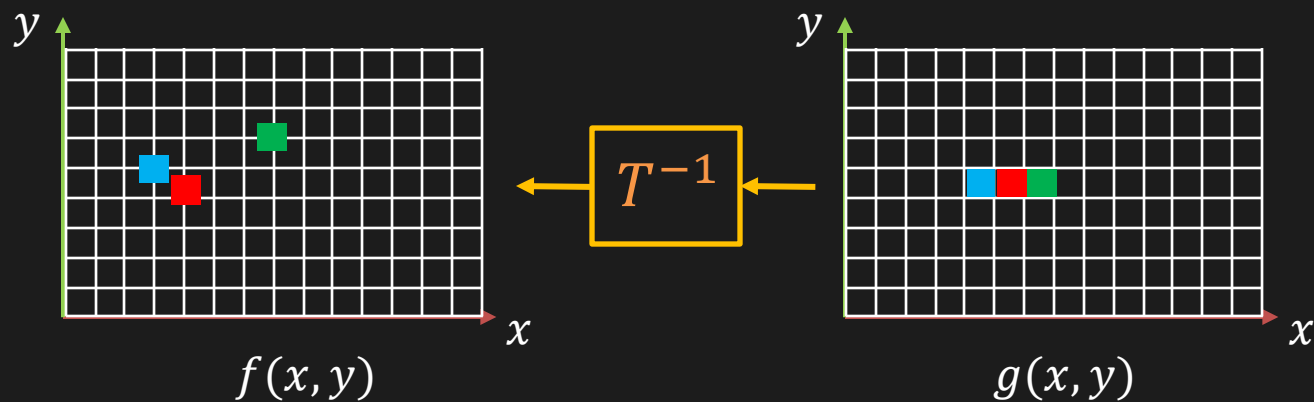
What if pixel lands in between pixels?
What if not all pixels in $g(x, y)$ are filled?

Can result in holes!

Backward Warping

Get each pixel (x, y) in $g(x, y)$ from its corresponding location $T^{-1}(x, y)$ in $f(x, y)$

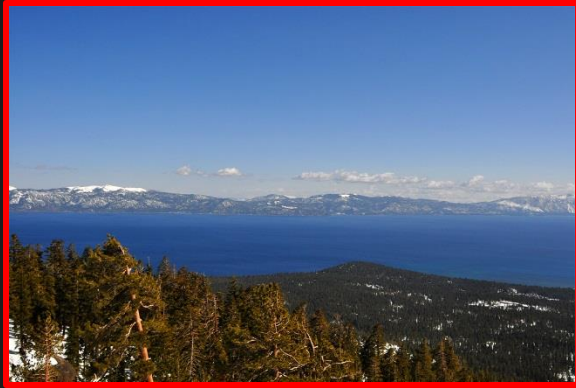
$$g(x, y) = f(T(x, y))$$



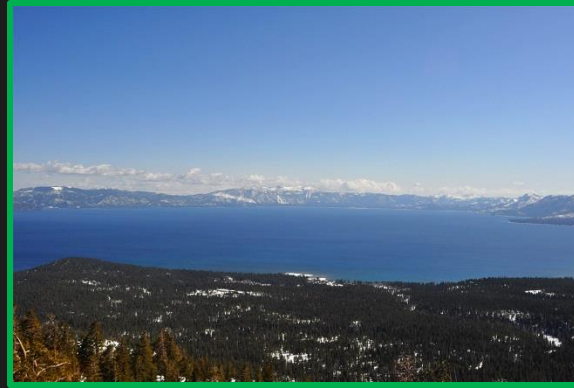
What if pixel lands between pixels?

No problem. Use **Nearest Neighbor** or **Interpolate** (Weighted Average of Neighbors)!

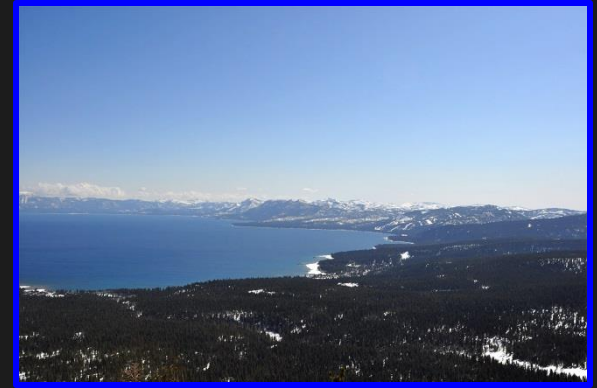
Image Alignment Process



Source Image 1



Source Image 2

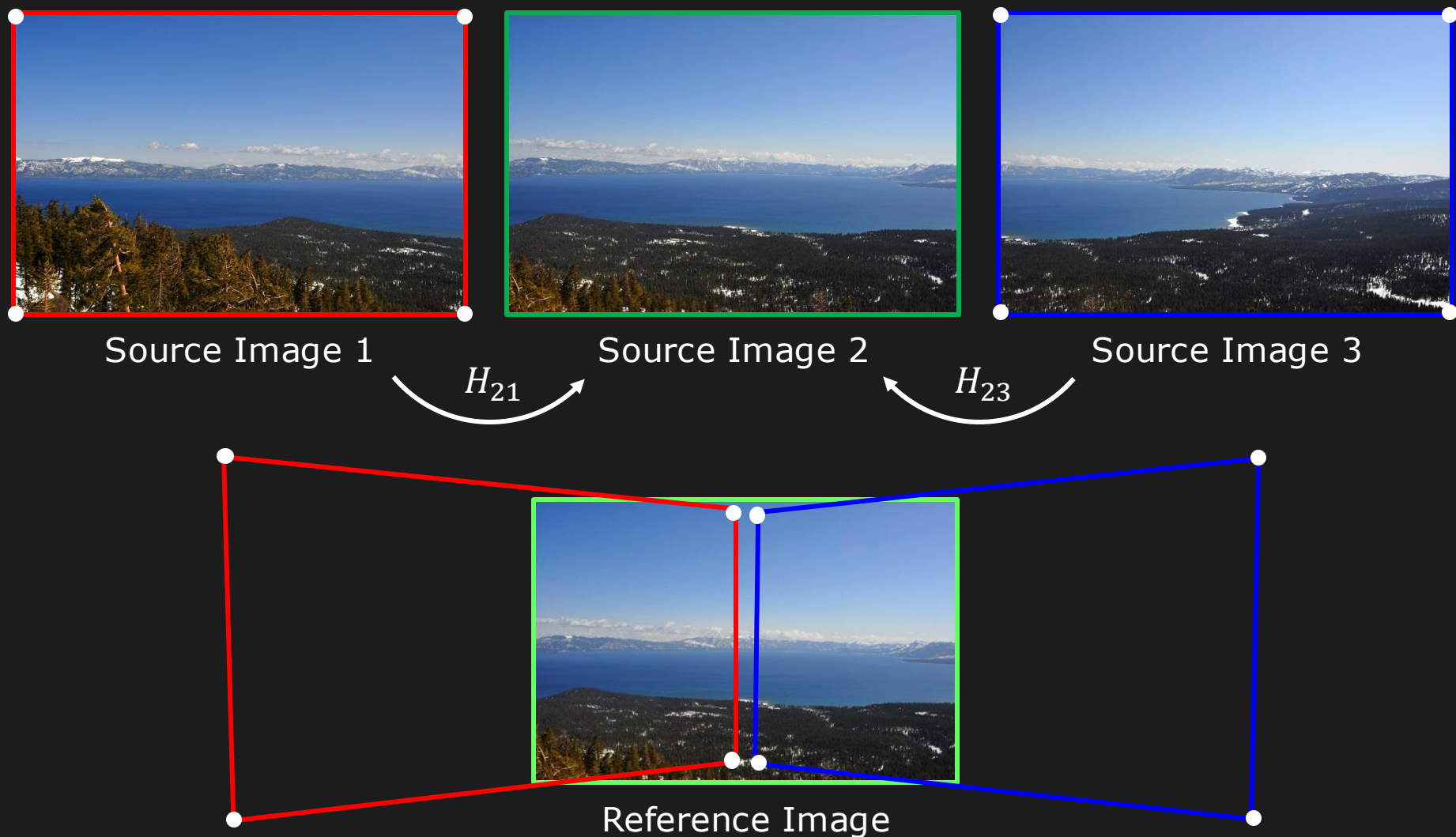


Source Image 3



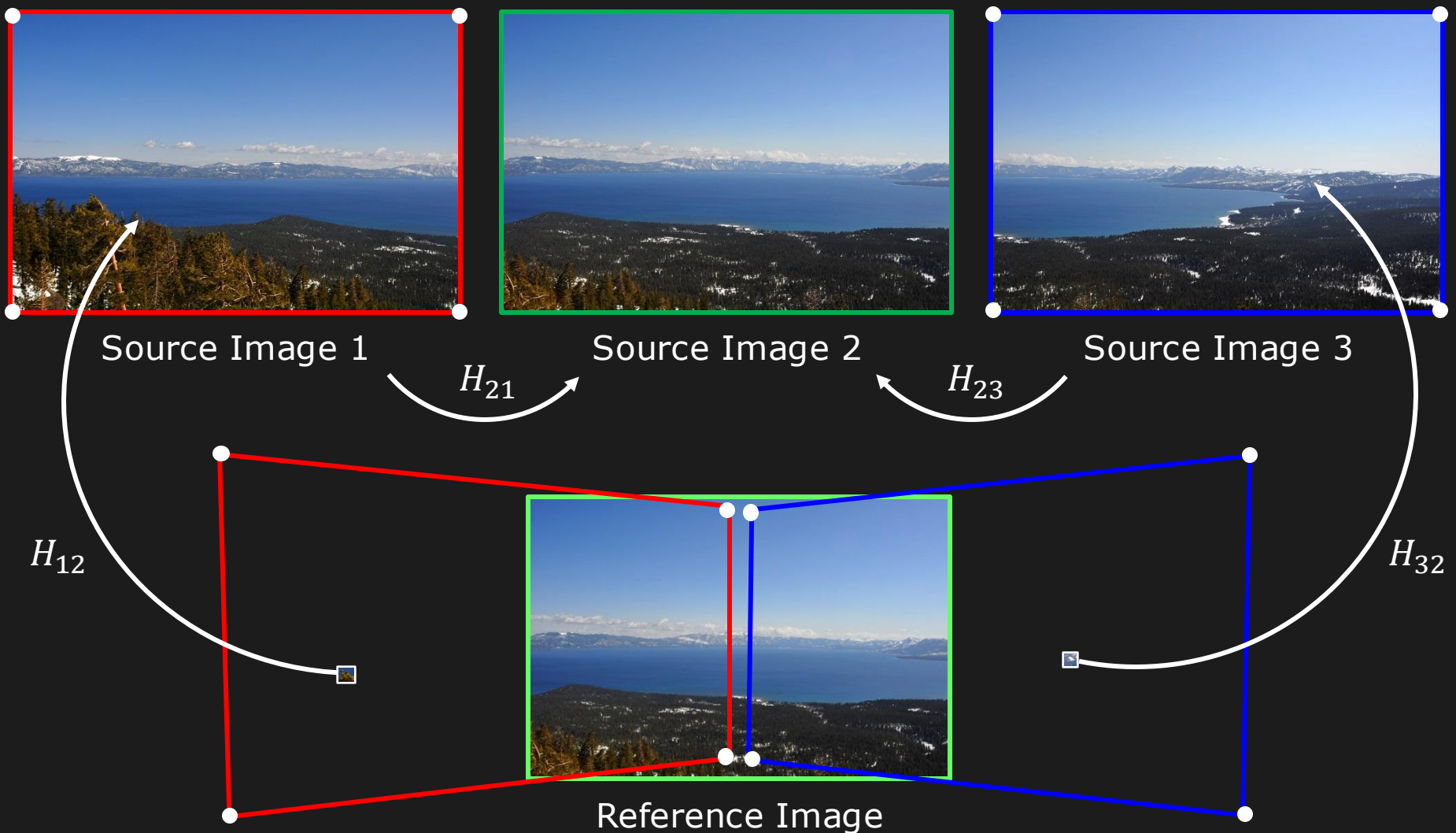
Reference Image
(Source Image 2)

Image Alignment Process



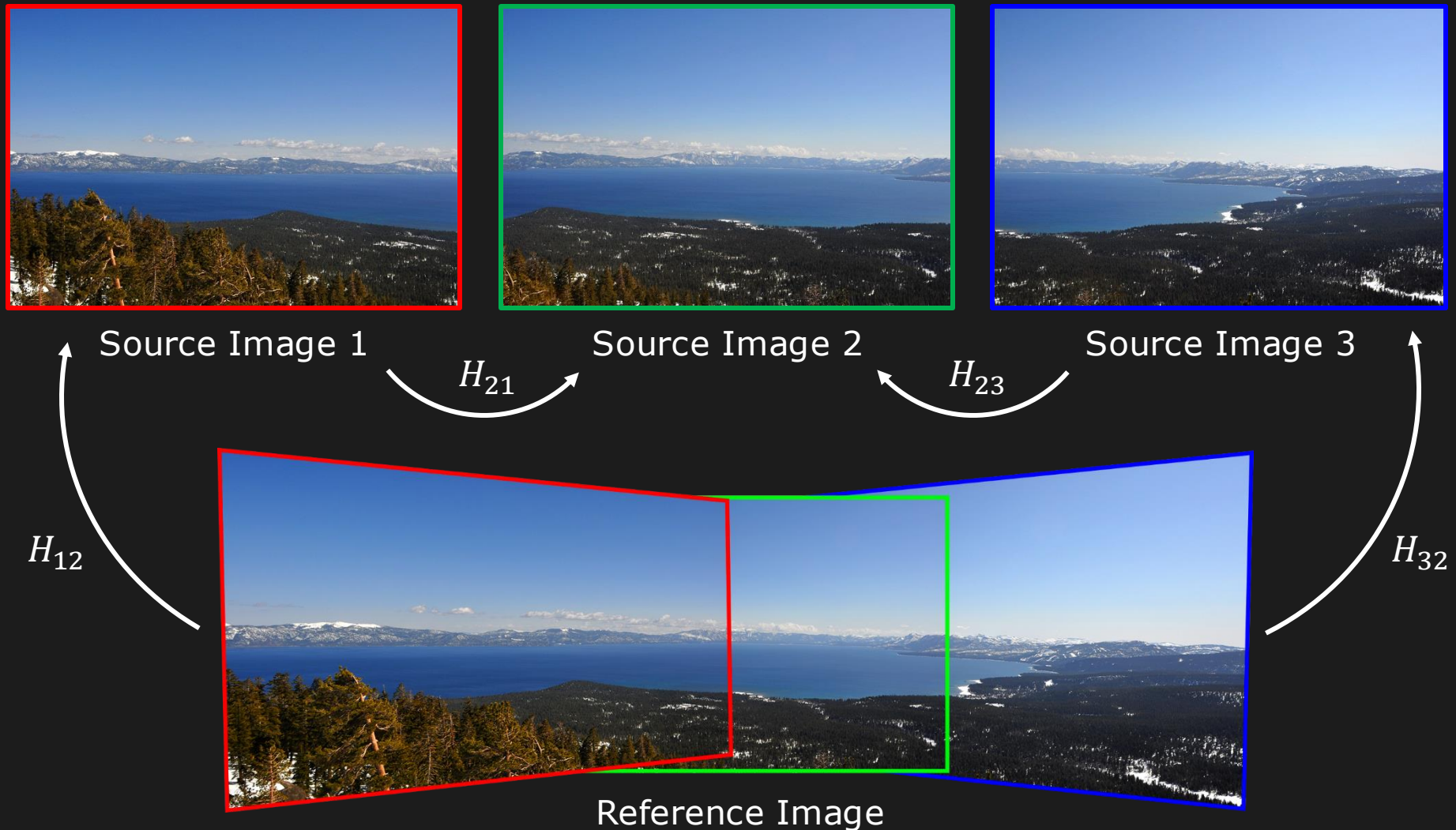
Compute the bounds of Image 1 and Image 3 in reference image space

Image Alignment Process



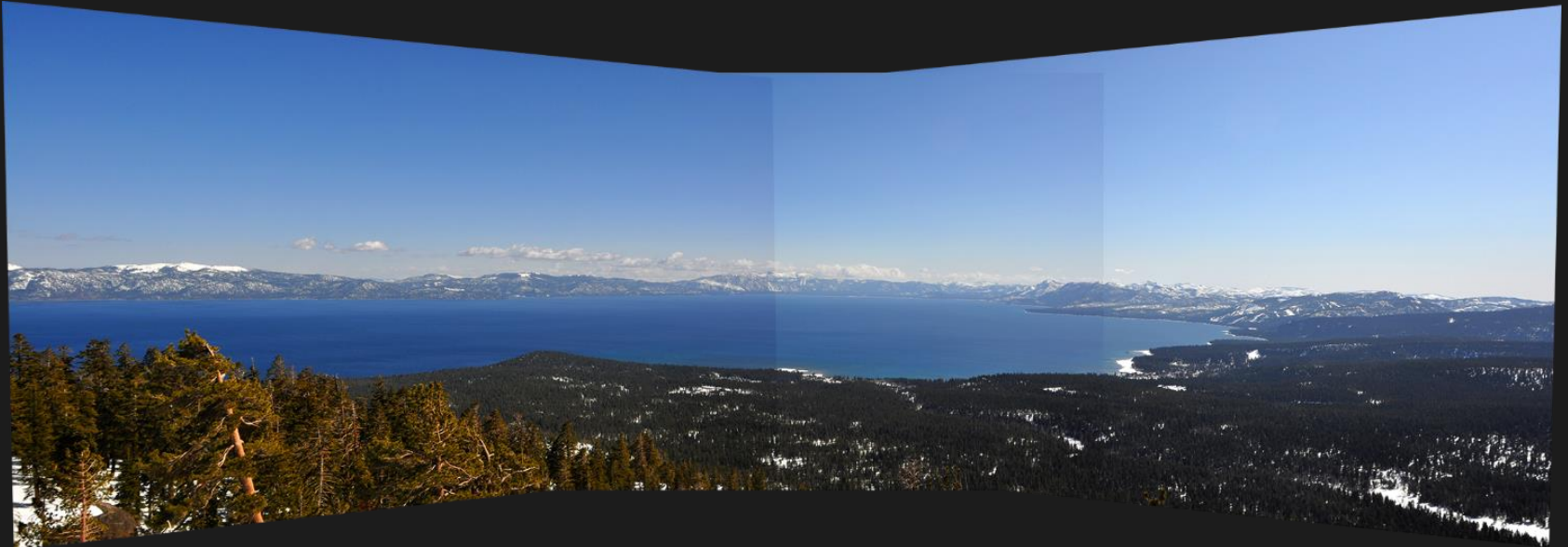
For each pixel within bounds, compute its location in source image

Image Alignment Process



For each pixel within bounds, compute its location in source image

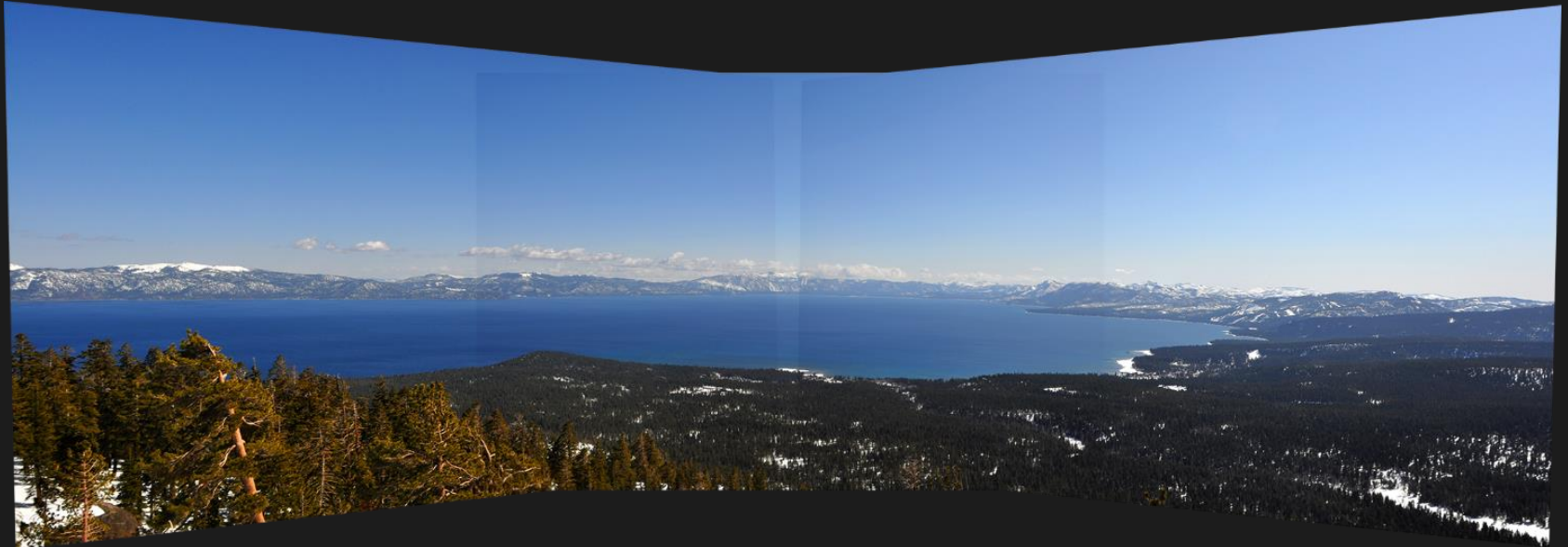
Blending Images



Overlaid Aligned Images

Hard seams due to vignetting, exposure differences, etc.

Blending Images: Averaging



Averaged Images

Seams still visible.

Blending Images



Image I_1

+

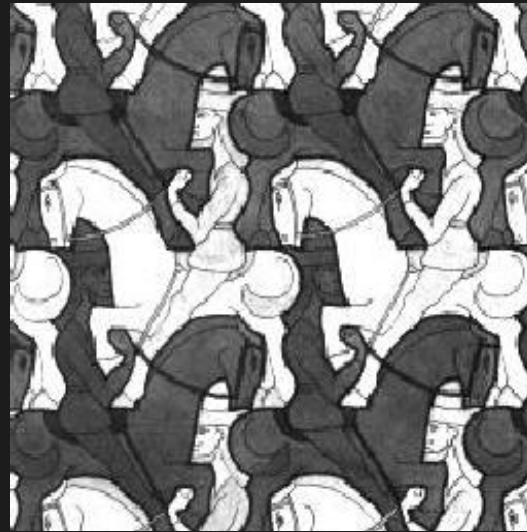
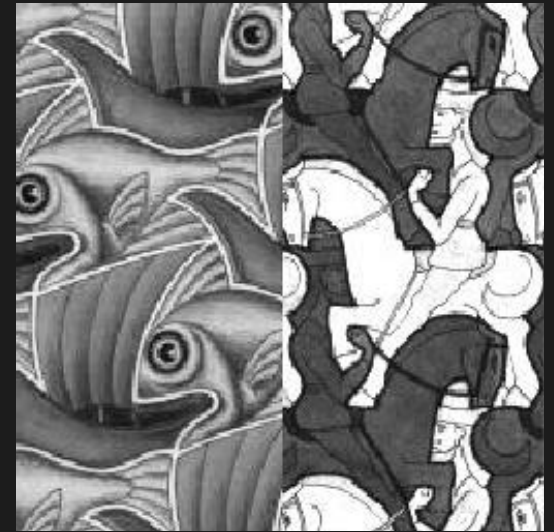


Image I_2

=



Hard overlay



Say we want to blend images I_1 and I_2 at the center.

Weighted Blending



Image I_1

+

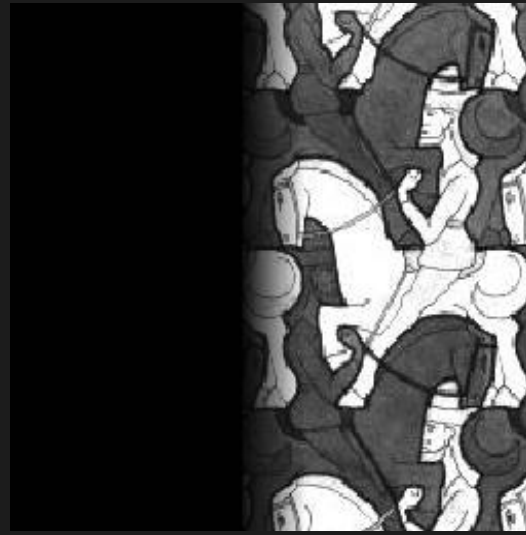
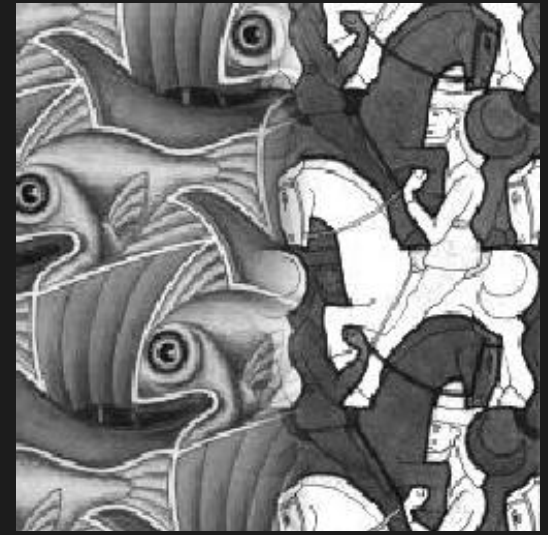


Image I_2

=



Blended Image

I_{blend}



$$I_{blend} = \frac{w_1 I_1 + w_2 I_2}{w_1 + w_2}$$

Weighted Blending

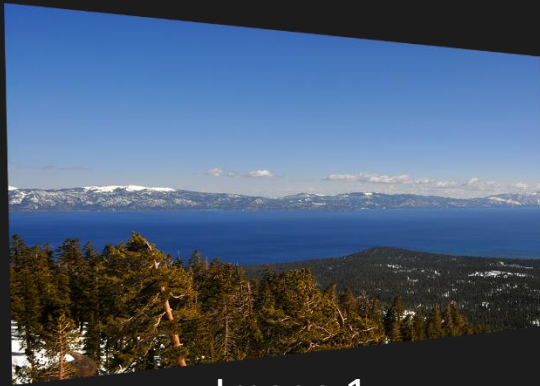


Image 1



Image 2

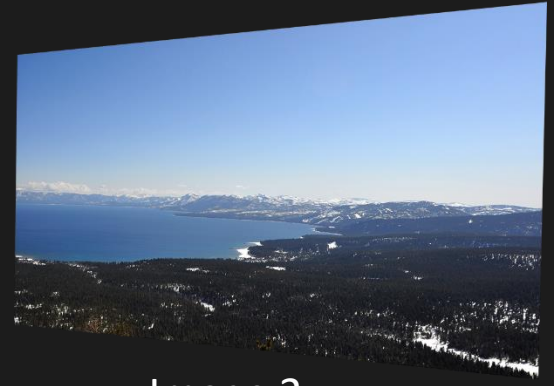
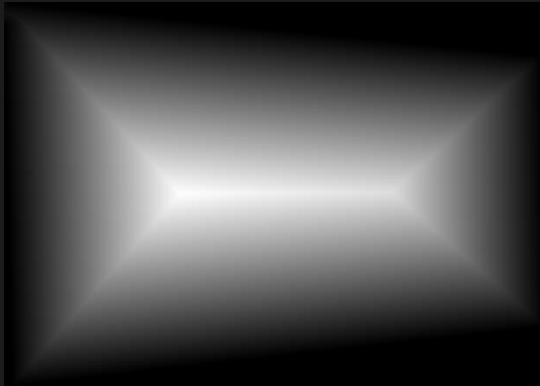


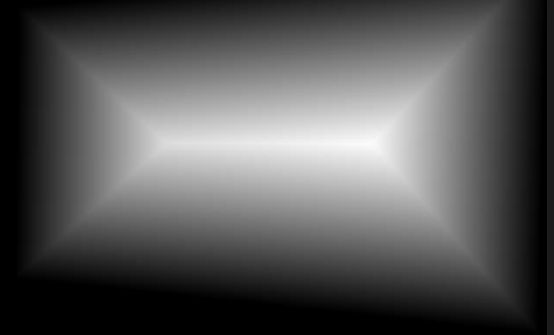
Image 3



Weight w_1



Weight w_2



Weight w_3

Pixels closer to the edge get a lower weight.

Ex: Distance Transform (`bwdist` in MATLAB).

Weighted Blending

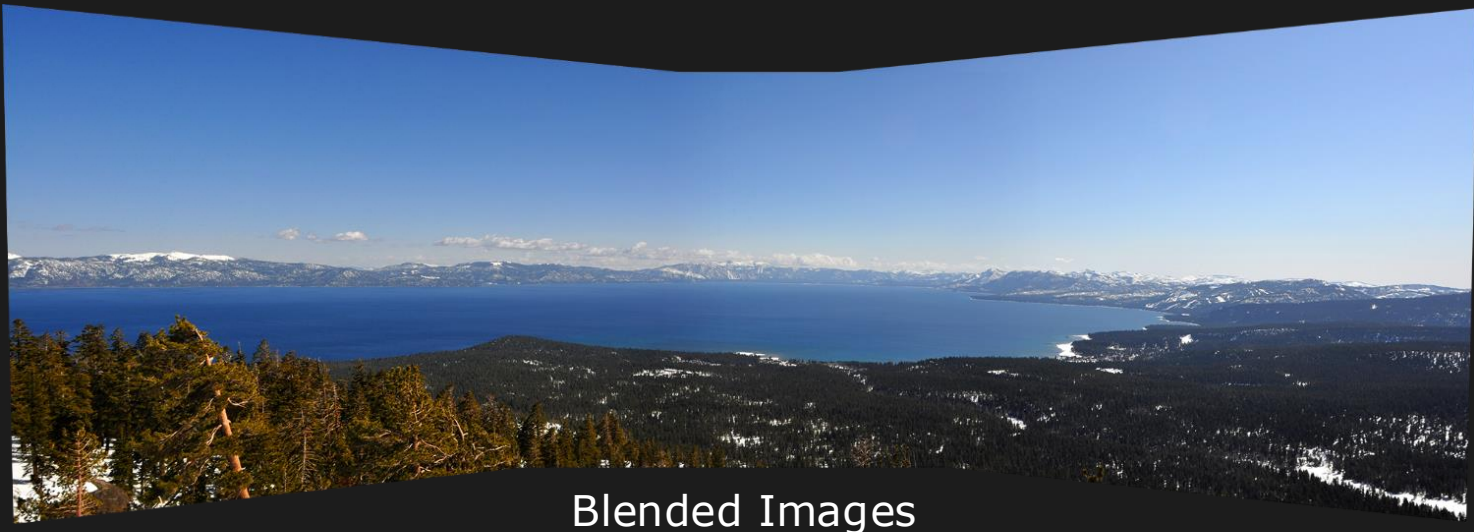
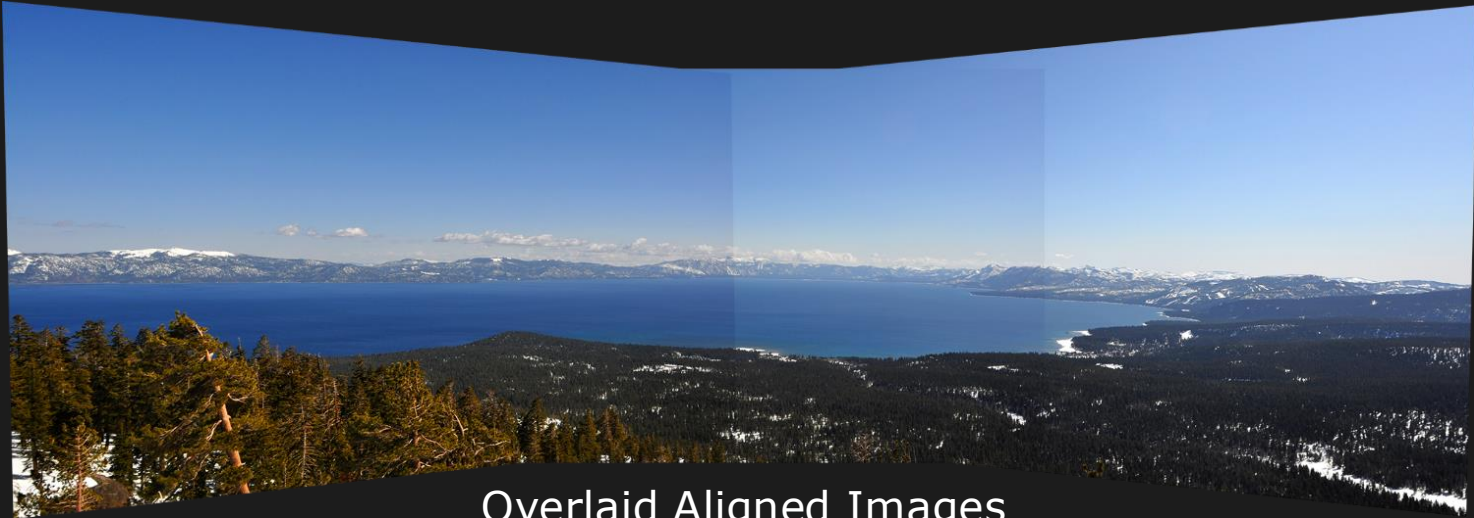
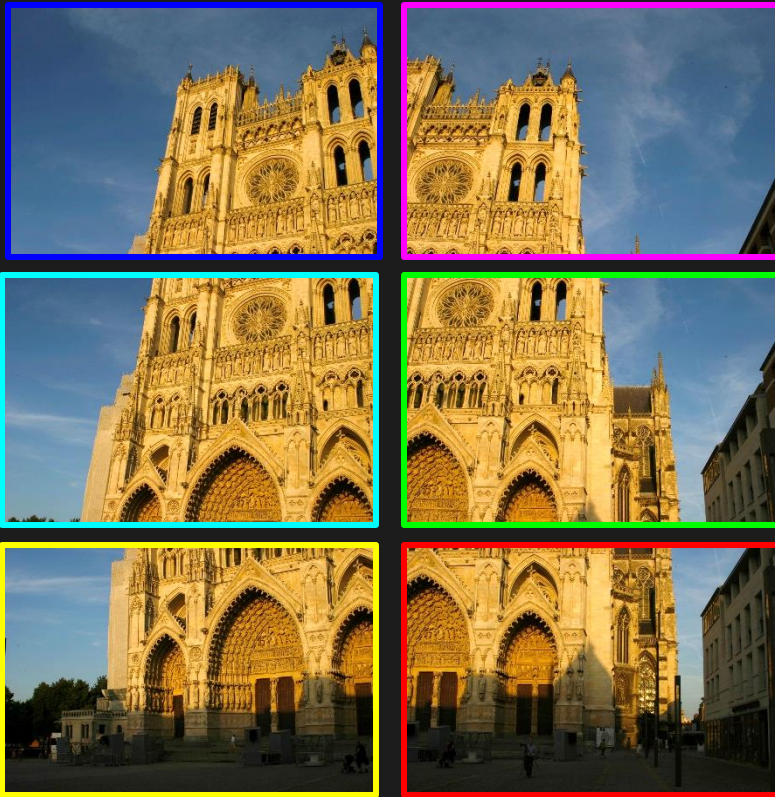
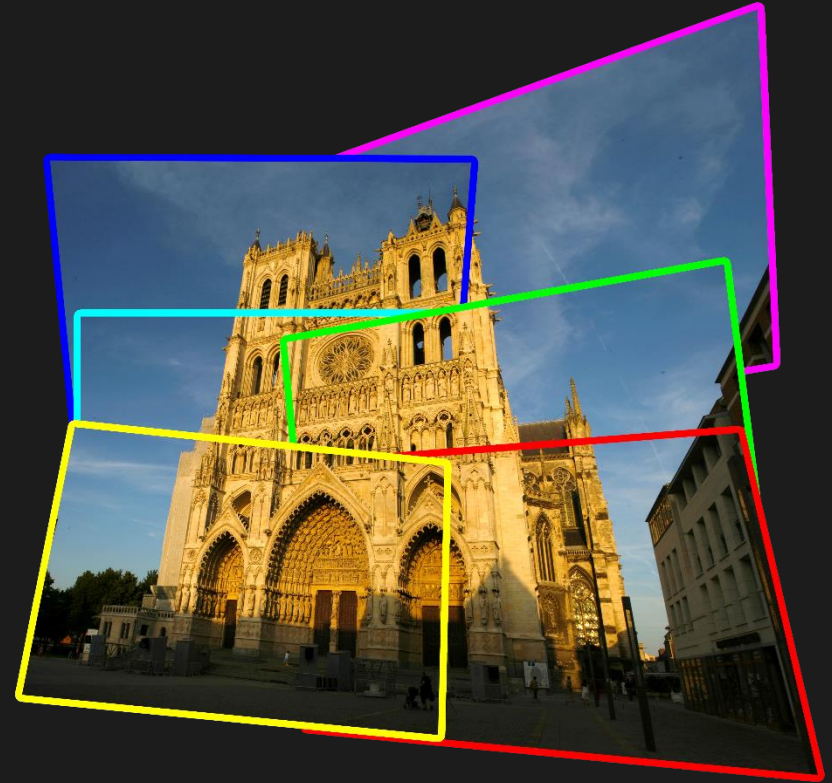


Image Stitching Example

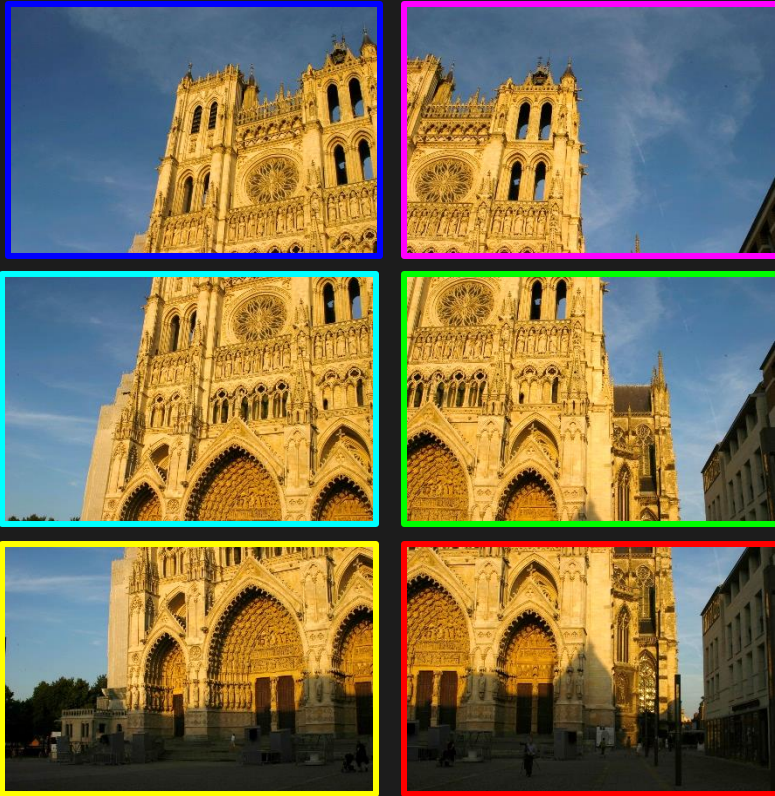


Source Images

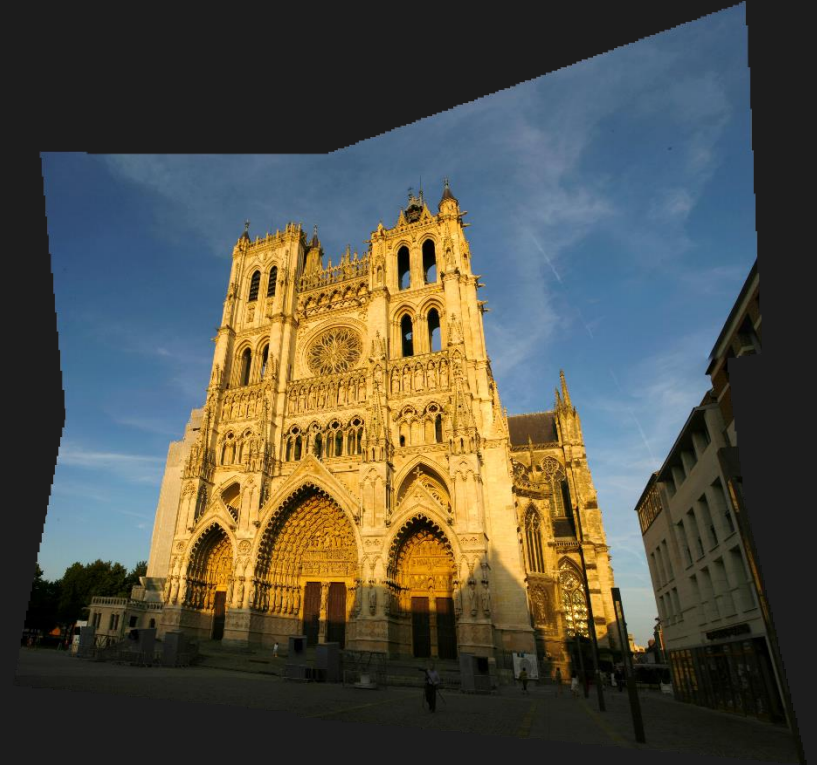


Aligned Images

Image Stitching Example



Source Images



Blended Images

References: Textbooks

Computer Vision: Algorithms and Applications (Chapter 2, 9)

Szeliski, R., Springer

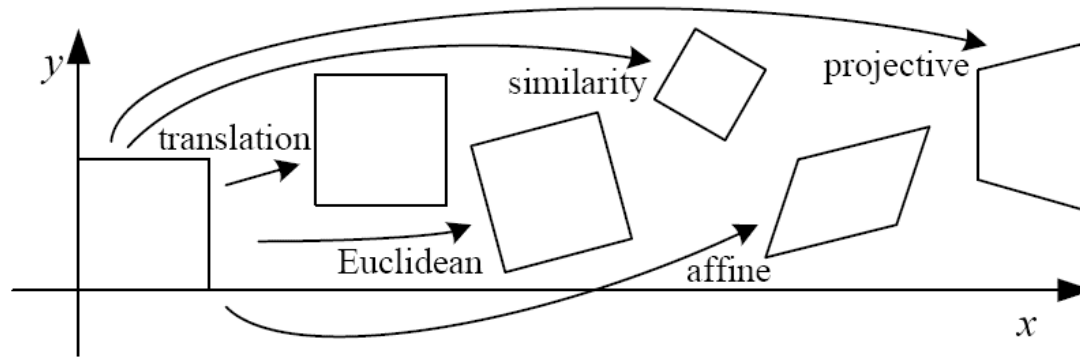
References: Papers

[Fischler 1981] Fischler M. A. and Bolles R. C. "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography", 1981.

Image Credits

- I.1 <http://www.flickr.com/photos/byspice/4577634277>
- I.2 <http://www.ptgui.com/examples/quicktour5/>
- I.3 Figure 2.4, Table 2.1, Computer Vision: Algorithms and Applications, Szeliski, R., Springer

Appendix A: Linear Transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Appendix B: RANSAC

How many iterations?

Suppose,

- p : probability or confidence with which we need the correct fit.
- α : probability of choosing an inlier while selecting the s samples.

If the s points are selected **independently**. Then,

Minimum number of iterations required:

$$N = \frac{\log(1 - p)}{\log(1 - \alpha^s)}$$

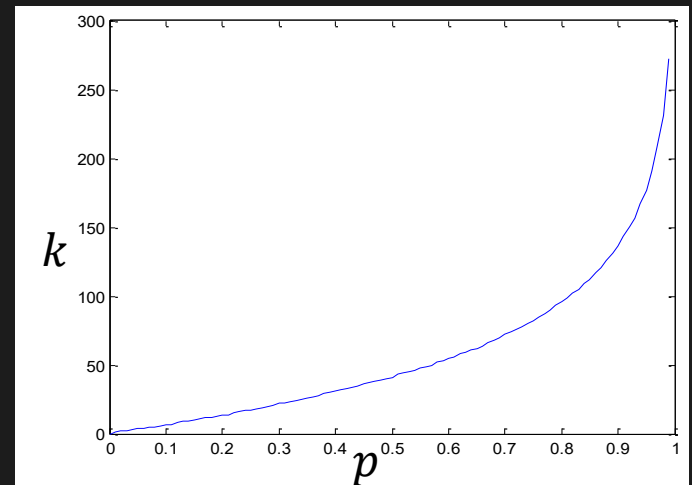
Appendix B: RANSAC

Minimum number of iterations required:

$$k = \frac{\log(1 - p)}{\log(1 - \alpha^s)}$$

	Proportion of inliers α						
s	95%	90%	80%	75%	70%	60%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Minimum number of iterations ($p = 0.99$)



$s = 8, \alpha = 60\%$